



Um guia completo sobre o Modelo Corporativo de DevOps



Versão 1.4
02/08/2023

TODAS AS INFORMAÇÕES AQUI CONTIDAS SÃO CONFIDENCIAIS.

Esse nível é atribuído às informações cujo conhecimento é do interesse de toda instituição, incluindo empregados CAIXA, prestadores de serviço, usuário Fábrica e menores aprendizes no exercício de suas atribuições. São informações necessárias ao desempenho de suas atribuições na CAIXA, não cabendo o seu conhecimento a empregados de empresas contratadas que não possuam nenhuma atribuição junto ao serviço ou produto contratado. Outras pessoas ou empresas deverão ser autorizadas pela **GEQTI** de acordo com a política vigente sobre divulgação de informações corporativas.

Este documento contém informações proprietárias.

HISTÓRICO DE REVISÕES

Data	Versão	Descrição	Autor
11/02/2022	1.0	Criação da primeira versão do documento.	Manuel Nunes Pereira
15/07/2022	1.1	Inclusão de: • política de controle de permissionamento para branches; • aprovação e complementação do pull request; • trigger para Integração Contínua; • ajustes gerais.	Manuel Nunes Pereira
30/12/2022	1.2	Alteração do tópico sobre Migração de Módulos Legados	Manuel Nunes Pereira
23/01/2023	1.3	Inclusão dos serviços de esteiras na nuvem	Manuel Nunes Pereira
02/08/2023	1.4	Inclusão de novas orientações sobre esteiras na nuvem	Manuel Nunes Pereira

Sumário

1.	<i>Introdução</i>	7
2.	<i>Conceitos</i>	8
3.	<i>Projeto DevOps Corporativo</i>	10
3.1.	Escopo	11
3.2.	O Modelo Agile-Devops	12
3.3.	Infra como código	13
3.4.	Ganhos na adoção do Modelo Corporativo	14
3.5.	Premissas	15
3.6.	modelos de pipelines	16
4.	<i>Migração</i>	19
4.1.	Passos da Migração	21
4.2.	Migração de Módulos Legados.	23
5.	<i>Arquitetura do Modelo Corporativo de DevOps</i>	25
6.	<i>Azure Devops Server</i>	28
6.1.	Coleções e Projetos	28
6.1.1.	Single Project	29
6.1.2.	Coleção projetos & projeto caixa	29
6.1.3.	Outras coleções	30
6.2.	Módulos & Recursos	31
6.2.1.	Azure overview	31
6.2.2.	Azure boards	31
6.2.3.	Azure Repos	32
6.2.4.	Azure Pipelines	32
6.2.5.	Azure Test Plans	32
6.2.6.	Azure Artifacts	33
6.2.7.	Marketplace de extensões	33
6.3.	Papéis, Times & Grupos	34
6.3.1.	Permissionamento no projeto caixa	34
6.3.2.	Principais papeis Nativos	34
6.3.3.	Desenvolvedor	35

6.3.3.1.	diferenças entre o papel contributor (nativo) e o papel de Desenvolvedor (customizado)	35
6.3.4.	Qualidade	37
6.3.5.	Suporte (Antigo SAI)	37
6.3.6.	Segurança	38
6.3.7.	Aprovador	38
6.3.8.	Operação	38
6.4.	Grupos & Times	39
6.4.1.	Grupos de comunidade no ADS	39
6.4.2.	Grupos no AD (Active Directory)	40
6.4.3.	Reuso dos Grupos de AD no clearcase	40
6.4.4.	Grupos Transversais	40
6.4.5.	Times	41
6.5.	Projetos no ADS	43
6.5.1.	Criar Projetos nas novas coleções	43
6.5.2.	Grupos e Perfis nativos	44
6.6.	Repositórios no ADS	48
6.6.1.	Padrões de nomenclatura	48
6.6.2.	Tipos de repositórios	48
6.6.3.	Criação de repositórios	49
6.6.4.	Espelhando repositórios	50
6.6.5.	Permissionamento para os repositórios	51
6.6.6.	Restringir acesso aos antigos repositórios	52
6.6.7.	Tipos de arquivos	53
6.6.8.	Políticas no uso de Branch	54
6.6.9.	Pull Request	55
6.6.9.1.	Como criar um pull request	56
6.6.9.2.	Como aprovar um pull request	57
6.6.10.	Controle semântico de versão	58
6.6.11.	Padrão para tags	59
6.6.12.	Criação das tags no ADS	60
6.6.13.	Controle semântico das pipelines	61
6.6.14.	Permissionamento para branches	62
6.7.	Esteiras de DevOps	64
6.7.1.	Pipelines de build	66
6.7.1.1.	Snapshot	66
6.7.1.2.	Artefatos como produtos	67
6.7.1.3.	Templates	68
6.7.1.4.	Criação da pipeline	69
6.7.1.5.	Criação de tarefas (tasks)	69
6.7.1.6.	Criação de grupos de tarefas (task groups)	69

6.7.1.7.	Controle de versão das tarefas e task groups	70
6.7.1.8.	Nomenclatura da pipeline	71
6.7.1.9.	Permissionamento da pipeline	71
6.7.1.10.	Localização da pipeline	72
6.7.1.11.	Execução da pipeline	73
6.7.1.12.	Acompanhamento dos logs	74
6.7.1.13.	Montagem da imagem	75
6.7.2.	Pipelines de release	75
6.7.3.	Variable groups	78
6.7.4.	Trigger para Integração Contínua	79
7.	Azure NA NUVEM (AZURE Devops Service)	81
7.1.	Organização CAIXA_AZURE	81
7.2.	Crerios para criação de projetos no CAIXA_AZURE	82
7.3.	Criação de projetos no caixa_azure	82
7.3.1.	Perfis & permissionamento no CAIXA_AZURE	84
7.3.2.	Criação de repositórios e pipelines no CAIXA_AZURE	85
7.3.3.	Tarefas de qualidade	86
7.3.4.	Atuações no modelo devops para nuvem	87
7.3.5.	Diferenças entre o modelo on premise e nuvem	87
7.3.6.	ORIENTAÇÃO OPERACIONAIS PARA DEVOPS NA nuvem	89
8.	Testes de qualidade	90
8.1.	Sonar	91
8.1.1.	Arquitetura	91
8.1.2.	Fluxo de execução	92
8.1.3.	Quality Gate	93
8.1.4.	Painel do projeto	94
8.1.5.	Analisando o log	95
8.2.	Teste Unitário e cobertura de código - Junit	97
8.3.	Testes de Segurança	100
8.4.	Fortify	102
9.	Repositório de binários	103
9.1.	Componentes	105
9.1.1.	Instalando JDBC Drivers disponíveis no binario.caixa	106
9.1.2.	Deployments adicionais JBOSS	107
9.2.	Nomenclatura de Repositórios	108

9.3. Caixa-group	108
9.3.1. Popular repositórios	109
9.3.2. Acesso externo	109
9.3.3. Expurgo	110
10. Indicadores & Indutor	111
10.1. Plano de Indicadores do ADS	111
10.2. Indutor DevOps	113
10.2.1. Níveis	114
10.2.2. Nível 1	114
10.2.3. Nível 2	115
10.2.4. Nível 3	115
10.2.5. Nível 4	116
10.2.6. Nível 5	117
10.2.7. Relatórios de nível de maturidade	117
11. Suporte	118
11.1. Requisição de inclusão ou migração de sistema no ADS:	118
11.2. solicitação de inclusão de sistema/módulo na nuvem	120
11.3. Solicitação de acesso as ferramentas OKD, Grafana e Graylog	121
11.4. Suporte a ferramental devops	121

1. INTRODUÇÃO

Este documento foi criado para ser referência do Modelo Corporativo de Devops para as esteiras automatizadas.

Além disso, ao centralizar informações sobre o Modelo Corporativo de DevOps, facilita o processo de gerenciá-las, atualizá-las e disponibilizá-las de maneira mais eficiente e acessível a todos os interessados.

O público-alvo deste documento são os diversos usuários, especialmente desenvolvedores (dev) e agentes de qualidade (qa), que ainda não estão totalmente familiarizados com este Modelo e para aqueles que desejam aumentar o seu conhecimento sobre os diversos assuntos aqui tratados. Portanto, este documento possui um caráter didático, apresentando-o de maneira estruturada e temática.

O Modelo Corporativo de Devops tem como portal, o [devops.caixa](#), que é uma interface web da ferramenta Azure Devops Server ou simplesmente, ADS.

Além do ADS, compõe o modelo corporativo o [fontes.caixa](#) (GitLab) e o [binario.caixa](#) (nexus).

É importante salientar que equipes de desenvolvimento que ainda não adotam as ferramentas aderentes ao modelo corporativo aqui elencadas nas suas esteiras de automação DEVEM solicitar tempestivamente a migração.

Esteiras baseadas em Jenkins, TFS, entre outras, não possuem suporte corporativo e estão em desuso.

2. CONCEITOS

Integração Contínua - prática de automatizar a integração de alterações de código de vários colaboradores e permitir que os artefatos gerados e testados sejam publicados em um repositório central de maneira contínua. No devops.caixa é representado pelo módulo de pipeline.

Implantação Contínua – prática de publicação de releases que usa testes e configurações de ambientes automatizados e de maneira contínua. No devops.caixa é representada pelo módulo de releases.

Entrega Contínua - uma série de práticas destinadas a garantir que o código possa ser implantado com rapidez e segurança na produção, fornecendo todas as alterações em um ambiente de trabalho e garantindo que as aplicações e serviços de negócios funcionem como esperado por meio de testes adequados. É o ciclo completo.

Artefato – Segundo a Engenharia de Software, artefato é um dos vários tipos de subprodutos produzidos durante o desenvolvimento de software. Artefatos podem ser documentos, arquivos executáveis, módulos etc. No contexto deste documento, são os arquivos binários produzidos pela pipeline.

Pipelines: consiste em uma série de etapas automatizadas a serem realizadas para a disponibilização de uma nova versão de um software.

Stage: Uma pipeline de release é organizada em stages. Na Caixa, deve ocorrer um stage para cada ambiente, como DES, TQS, HPM e PRD. Os stages são serializados e podem ser interligados, garantindo um fluxo contínuo.

Task Groups: Conjunto de tarefas afins que compõe uma pipeline e/ou stage. Ex: Task Group para build, scan do Sonar, etc...

Variable Groups: Conjunto de variáveis associadas as pipelines ou stages. As variáveis de configuração de um determinado ambiente requerem uma variable group específica de vinculação a cada stage pertinente.

Libraries – No contexto deste documento, libraries são grupos de variáveis ou arquivos de segurança e similar as variables groups, podem ser vinculados as pipelines e aos stage de cada ambiente.

Override – ação de sobrepor as políticas de branch previamente estabelecidas

Políticas de Branches – as branches master e develop (de integração) devem ser protegidas a aplicadas políticas de controle de acesso. O pull request para a principal branch de desenvolvimento (develop) é altamente recomendado, assim como a Integração Contínua ser automática a cada aprovação de pull request para a develop.

Pull Request – Solicitação de merge de um commit para uma branch principal. O Pull request é uma oportunidade para (i) comunicar que há uma nova versão candidata; (ii) promover a integração do código e sincronização dos commits numa branch develop; (iii) invocar uma terceira parte para controle e andamento das versões dos códigos quando há paralelismos; (iv) provocar um momento de oportunidade para análise do código por um Agente de Qualidade e (v) solicitar uma autorização quando a pipeline de build quebrar por déficit de qualidade, quando configurado o modelo de autorização.

3. PROJETO DEVOPS CORPORATIVO

DevOps é uma cultura de desenvolvimento que aproxima as áreas de desenvolvimento e operações com o objetivo de convergir pessoas, processos e tecnologias para criar e fornecer continuamente valor aos clientes.

O que o DevOps significa para as equipes? O DevOps permite que funções anteriormente isoladas – desenvolvimento, operações de TI, engenharia da qualidade e segurança – atuem de forma coordenada e colaborativa para gerar produtos melhores e mais confiáveis. Ao adotar uma cultura de DevOps, somada as práticas e ferramentas de DevOps, as equipes ganham a capacidade de responder melhor às necessidades dos clientes, aumentar a confiança nos aplicativos que constroem e cumprir os objetivos de negócio de maneira ágil.

A CAIXA realizou várias iniciativas de DevOps nos últimos anos, muitas delas baseadas em ferramental, e isoladamente, com patrocínios locais.

Boa parte destas iniciativas eram mais voltadas a automação de fluxos de trabalho e começaram a surgir várias instâncias de ferramentas como Jenkins, gitlab, nexus, todas mantidas e gerenciadas de maneira descentralizada.

O projeto de um Modelo Corporativo de Devops surgiu como uma necessidade de implantação de políticas convergentes de DevOps, coordenadas pela matriz e que adotasse padrões aderentes as normas e melhores práticas de arquitetura, processo e infraestrutura.

3.1. ESCOPO

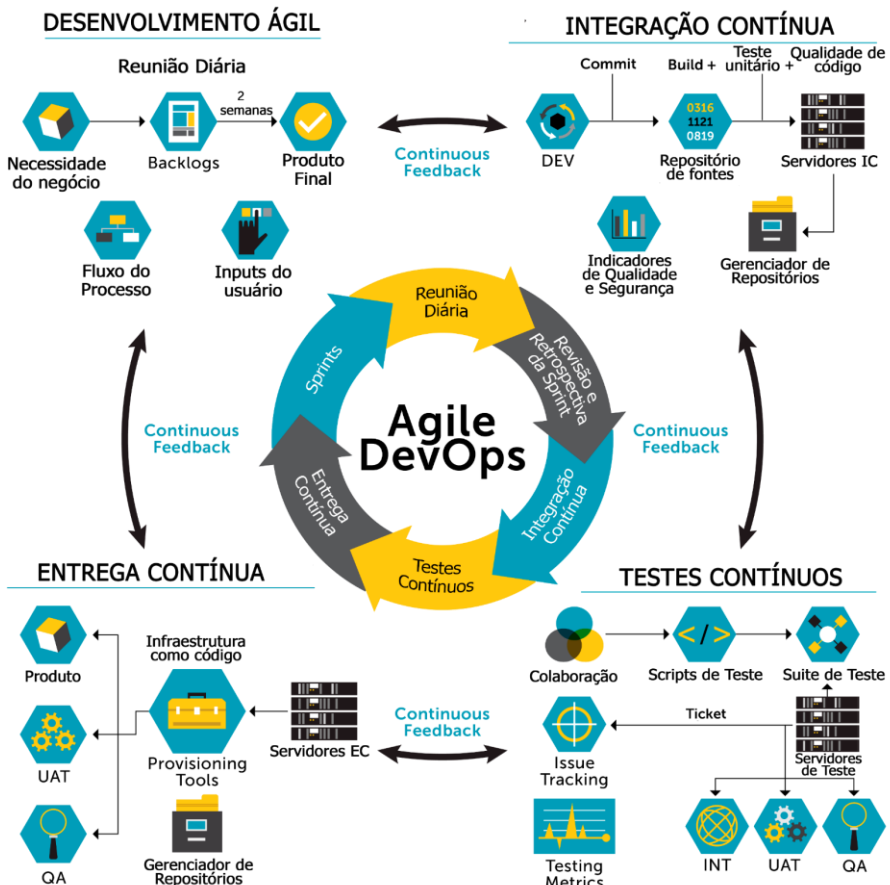
O projeto do Modelo Corporativo de DevOps apresenta os seguintes objetivos:

- Implantação estruturada de uma cultura DevOps para toda a CAIXA.
- Convergir as diversas iniciativas de DevOps para obter nas esteiras automatizadas uma melhor aderência as práticas, padrões, processos e normas vigentes da CAIXA.
- Manter um modelo de DevOps gerenciável, suportável e escalável.
- Aproximar as diversas áreas interessadas para colaborar com toda a cadeia automatizada da Entrega Contínua, como segurança e governança.
- Criar esteiras baseadas em templates mantidos e atualizados que automatizam a criação de ambientes de TI, testes e implantação de software de forma padronizada, sustentável, escalável e monitorada.

3.2. O MODELO AGILE-DEVOPS

Atualmente, não há como dissociar DevOps de Metodologias Ágeis de Desenvolvimento. Não apenas por serem as práticas mais adotadas e referenciadas de mercado, mas também por serem convergentes e complementares.

O Projeto de DevOps Corporativo é totalmente integrado ao conceito de Esteiras Ágeis.

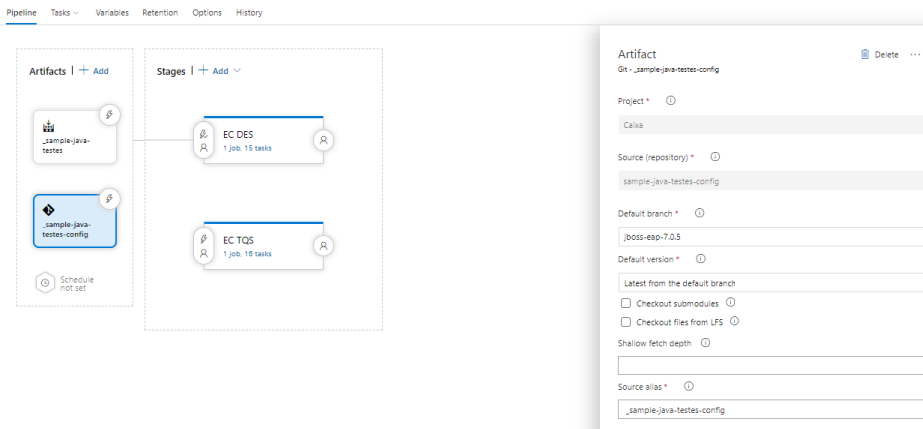


3.3. INFRA COMO CÓDIGO

Infraestrutura como código (IaC) é o gerenciamento e provisionamento da infraestrutura por meio de códigos de maneira automatizada, versionada e replicável, em vez de processos manuais.

IaC é parte fundamental do modelo corporativo de DevOps para integração e implantação contínuas (CI/CD) nas esteiras ágeis.

As configurações são mantidas versionadas em repositórios com o sufixo -config ou -cfg e são utilizadas nas pipelines de release.



Um modelo corporativo não seria viável se os times de desenvolvimento e de operações implantassem aplicações ou configuram ambientes de maneira diferente, mesmo que haja uma abordagem DevOps. É necessário adotar o mesmo modelo para ganhos efetivos de alinhamento das equipes de desenvolvimento e de operações e assim realizar entregas de maneira contínua com menos erros e inconsistências.

3.4. GANHOS NA ADOÇÃO DO MODELO CORPORATIVO

Os benefícios na adoção de uma cultura DevOps são inúmeros e com a adoção de um Modelo que o mantenha exequível nos permite obter:

- Aderência aos padrões vigentes na CAIXA;
- Orientação na adoção das melhores práticas;
- Adoção de ferramentas largamente conhecidas;
- Suporte operacional em diversos níveis;
- Testes e relatórios contínuos de qualidade e segurança;
- Escalabilidade no uso;
- Plataforma única (`devops.caixa`) para colaboração efetiva dos times e demais interessados;
- Redução do risco e custo operacional;
- Automação: compilação/*deploy*/testes/monitoração/Log.
- Tempo e facilidade de *rollback*.
- Promoção do uso do *pull request* para integração contínua e controle das *branches* de desenvolvimento e master;
- Integração, publicação e entrega contínua das soluções desenvolvidas;
- Monitoração e rastreamento da aplicação e infra, entre outros.

3.5. PREMISSAS

Para que sistemas possam migrar seus módulos para adoção de **Integração Contínua** no devops.caixa, é necessário que as seguintes premissas sejam atendidas:

- 1) O repositório de códigos-fonte DEVE ser GIT, preferencialmente fontes.caixa, onde a importação é muito simples, porém é possível migrar de qualquer repositório.
- 2) Preferencialmente, cada repositório será vinculado a uma pipeline.
- 3) As dependências em suas respectivas versões DEVEM estar disponíveis no Nexus (binarios.caixa).
- 4) Ter disponível o DAS ou documento similar que represente a arquitetura da aplicação/sistema para orientar o suporte na correta configuração das pipelines.
- 5) Informar o nome da comunidade ao qual os módulos estão vinculados, assim como os nomes das fábricas contratadas para criação de grupos e times.
- 6) Informar os Grupos de AD para as vinculações com os times os times que atuarão no ADS.
- 7) Deve-se ter compreensão dos parâmetros atuais necessários para aprovação no Quality Gate do SonarQube e demais ferramentas de testes para ter um processo de Integração Contínua sem quebra.
- 8) Deve-se adotar as tecnologias para compilação ou criação de artefatos previstos na pipeline, como o Maven. Caso não, será necessário a “mavenização”.

3.6. MODELOS DE PIPELINES

Além das premissas para uso do processo de Integração Contínua no devops.caixa, algumas ações são necessárias para o uso da **Implantação Contínua** no devops.caixa:

- 1) Ter um repositório dedicado para itens de configuração para cada módulo.
- 2) Mapear as configurações e as variáveis de ambiente para configurações dos diversos ambientes (DES|TQS|HMP|PRD) e mantê-las atualizadas nas variables groups associadas a cada stage da pipeline de release.
- 3) Mapear das portas e conexões que fazem parte do ecossistema da aplicação.

Além das ações, é importante avaliar se há modelos de pipelines disponíveis no azure.devops que usa tecnologias compatíveis com a sua necessidade.

Aqui seguem alguns exemplos para ilustrar compatibilidades com modelos padrões de pipelines para ambientes criados no OKD, reforçando que estas informações estão atualizadas apenas na data da publicação deste guia e que muito provavelmente estarão desatualizadas no decorrer do tempo. A lista completa de compatibilidades atualizadas para as pipelines de containers que rodam OKD e VM estão disponíveis na wiki do projeto CAIXA no devops.caixa.

JBOSS EAP

Nome imagem Docker: jboss-eap

Versões suportadas: 6.4.2x, 7.0.x, 7.1.x, 7.2.x, 7.3.x (para utilizar a versão 11 do JDK adicionar o texto -jdk11 ao final do nome da imagem. ex: 7.1.5-jdk11)

Versão dos componentes de compilação:

Controlado pela variável `JAVA_VERSION` nas pipelines de Java Jboss Container

`jdk1.7.0_79` `jdk1.8.0_221` `jdk-11.0.11` e `open-jdk-11`

Valor default é: `jdk1.8.0_221`

NODEJS

Nome imagem Docker: `nodejs`

Versões suportadas `release`: 6, 8, 9.4.0, 10.16, 10.16-oracle, boron, carbon, dubnium e stretch

Versão dos componentes de compilação:

Controlado pela variável `NODE_VERSION` nas pipelines de NodeJS e Angular

10.x e 12.x

ANGULAR

Versão dos componentes de compilação:

A versão do Angular deve ser declarada no `package.json` da aplicação

Nome imagem Docker: `nginx`

Versão suportada: 1.12, 1.18

PHP

Nome imagem Docker: `php`

Versões suportadas: 7.0, 7.2.4e 7.3

QUARKUS

Nome imagem Docker: `quarkus-java-binary-s2i`

Versão suportada: `java-11-openjdk`

OPENLIBERTY

Nome imagem Docker: `open-liberty-s2i`

Versão suportada: 20.0.0.6

APACHE HTTPD

Nome imagem Docker: `httpd-24-rhel7`

Versão suportada: 2.4-cef

.Net Core

Nome imagem Docker: dotnet-runtime

Versões suportadas: 2.1.817, 2.2.402, 3.1.410 e 5.0.301

4. MIGRAÇÃO

Nesta seção será apresentado o passo a passo para incluir ou migrar módulos para o modelo corporativo e poder se beneficiar dos ganhos do Modelo.

Quando se trata de novos sistemas/módulos que utiliza tecnologias compatíveis com os modelos de pipelines padrões, a inclusão no modelo corporativo é **IMEDIATA E OBRIGATÓRIA**.

Quando se trata de um módulo já existente, as questões a serem analisadas é se as premissas foram atendidas e se há compatibilidades para a migração para o devops.caixa. Em caso positivo para ambas as afirmações, a inclusão no modelo corporativo é **OBRIGATÓRIA**, porém, não precisa ser imediata, basta ter uma data de migração acordada entre a comunidade e a GECEQ.

Para incluir um módulo no devops.caixa, basta abrir uma requisição no serviços.caixa, conforme descrito na seção SERVIÇOS.

Se houver restrições de premissa ou compatibilidade, ainda é possível se beneficiar do Modelo Corporativo, como será abordado na seção “Migração de módulos não compatíveis.”

No momento de solicitação do serviço de inclusão/migração dos módulos devem ser informados:

- 1) a comunidade,
- 2) a sigla do sistema,
- 3) o(s) módulo(s),
- 4) a Fábrica de Software que suporta o sistema,
- 5) pessoa para contato,
- 6) o(s) repositório(s) git de origem (no caso de migração),
- 7) as tecnologias utilizadas pelos módulos e

- 8) se o uso é para Integração Contínua ou Entrega Contínua (ciclo completo). se o uso for para entrega contínua:
- 9) a compatibilidade de uso para container (ou VM)
- 10) Documento de Arquitetura (DAS ou similar) que informe:
 - endereços dos servidores (lista de backend's),
 - usuários para acesso aos ambientes de DES , TQS, HMP,
 - driver's que serão utilizados,
 - configurações de Data Sources e
 - demais configurações necessárias que contemplará cada um dos ambientes.

Após a abertura de requisição do serviço, um técnico especializado entrará em contato com a pessoa indicada para proceder o início do processo de inclusão/migração.

É importante destacar que as configurações a serem criadas nas esteiras ágeis, em especial, as variables groups, são fontes valiosas de informação da arquitetura do sistema/módulo para referência do próprio time ou demais interessados. Ela é mais importante do que a própria documentação pregressa, uma vez que são fontes diretas e continuamente atualizadas.

4.1. PASSOS DA MIGRAÇÃO

A imagem abaixo representa os passos realizados durante o processo de inclusão/migração de módulos no devops.caixa.



Pontos de Atenção:

A criação dos grupos e times deve seguir as orientações descritas na seção “Grupos e Times”

A criação/importação dos repositórios de fontes deve seguir as orientações descritas na seção “Criação de Repositórios”

A criação da pipeline de Build deve seguir as orientações descritas na seção “Criação da Pipeline”

A criação da pipeline de Release deve seguir as orientações descritas na seção “Criação da Pipeline de Release”

A criação das Variables Groups deve seguir as orientações descritas na seção “Criação das Variables Groups”

Ao final das criações e/ou configurações de todos os elementos envolvidos no escopo da requisição, deve-se solicitar as liberações de portas e firewall no portal.infra.

As informações de uso do portal estão disponíveis na wiki do projeto CAIXA.



infra.devops
Versão 1.7

infra.devops - login

Matrícula:
Digite sua matrícula

Senha:
Digite sua senha

ENTRAR

4.2. MIGRAÇÃO DE MÓDULOS LEGADOS.

Se o módulo a ser migrado ATENDE as premissas indicadas, mas não há compatibilidade (ainda) com os modelos de pipeline atualmente disponíveis para rodar a aplicação nos ambientes virtualizados, ainda assim é possível solicitar a migração.

Portanto, é possível oferecer aos repositórios e pipelines de build todos os ganhos previstos, como políticas de branches, integração contínua, testes automatizados e publicação do artefato no binário.caixa, mesmo para pipelines que não tenham tecnologias compatíveis com os atuais modelos de pipelines disponíveis no devops.caixa.

Neste caso, a solução de referência é publicar nos atuais ambientes utilizados pelo time, simplificando ao máximo a pipeline de release.

Esta opção de migração deve ser realizada prioritariamente para esteiras baseadas na tecnologia Jenkins ou demais esteiras automatizadas que não utilizam o devops.caixa.

No momento que houver compatibilidade, o time de desenvolvimento poderá solicitar a migração integral, ou seja, com pipelines capazes de criar e configurar ambientes automaticamente.

Os módulos legados terão repositórios, pipelines de build e release mantidos no projeto CAIXA e estarão submetidos ao mesmo padrão de governança dos demais sistemas e módulos.

Na figura abaixo, um exemplo de uma pipeline de release simplificada

EC DES
Deployment process

Agent job
Run on agent

 **Recuperando URL Pacote Nexus**
Bash

 **Recupera Pacote**
Bash

 **deploy package domain**
Bash

 **deploy package standalone**
Disabled: Bash

5. ARQUITETURA DO MODELO CORPORATIVO DE DEVOPS

A arquitetura projetada para suportar as esteiras ágeis do DevOps Corporativo é centrada no **ADS**, Azure Devops Server, ferramenta que orquestra as tarefas de automação para o processo de Integração e Entrega Contínua através das suas pipelines de build e release.



O ADS possui a tecnologia **GIT** embarcada, portanto, os times podem fazer uso desse módulo para gerenciar as versões dos seus códigos-fontes e demais scripts de configuração. Além do GIT embarcado, o **GitLab** acessado em fontes.caixa também é um repositório

corporativo e faz parte deste projeto para módulos que ainda não migraram para o devops.caixa.

Além dos repositórios de fontes, há o repositório de binários, para guarda de dependências, artefatos gerados, releases e demais componentes que se façam necessários. O repositório de binários utilizado é o **Nexus** e é acessado pela url: `binario.caixa`.

Existem rotinas automatizadas de testes que analisam a qualidade e segurança dos códigos implementados. As ferramentas **SonarQube** e **Fortify**, respectivamente, fazem a análise estática de código e, caso o processo de verificação não atenda aos requisitos mínimos definidos, a esteira “quebra” e não realiza qualquer publicação e, portanto, não gerando nenhum artefato. Não há como reativar a esteira, exceto realizando as correções necessárias para atingir os parâmetros de qualidade definidos numa nova versão, ou, excepcionalmente, com autorização justificada.

Maven é a principal ferramenta de automação de compilação utilizada pelas esteiras ágeis, principalmente para aplicações escritas em Java. O Maven utiliza um arquivo XML (POM) para descrever como a aplicação será montada, suas capturas das dependências hospedadas no `binarios.caixa`, a ordem de compilação, diretórios e plug-ins necessários.

O **Ansible** é uma ferramenta que utiliza comandos via protocolo ssh para gerenciamento, provisionamento, configurações, orquestração e demais comunicações em máquinas remotas. A montagem de seu playbook é uma lista de tarefas de automação a serem executadas em formato YAML. O Ansible é a principal tecnologia utilizada para IaC nas esteiras ágeis.

OKD é a principal ferramenta utilizada para gerenciamentos dos ambientes criados em container. Ela auxilia no processo de orquestração de containers de maneira independente da plataforma na qual os containers serão executados. OKD um acrônimo para

“Origin Kubernetes Distribution” e foi criado pela Red Hat para que existisse uma distribuição do Kubernetes mais otimizada para processos tradicionais de Integração Contínua e Implantação Contínua.

Grafana é a ferramenta escolhida para o monitoramento das aplicações desenvolvidas e publicadas pelas esteiras ágeis em seus ambientes. Ela fornece tabelas, gráficos e alertas para todos os interessados, o que permite qualquer desvio de desempenho e se o uso de recursos está dentro dos parâmetros esperados. O Grafana pode ser configurado para acessar diversos tipos diferentes de base de dados como o Prometheus e o Elasticsearch, entre outros.

Graylog é a ferramenta escolhida para centralizar, consultar, ordenar e classificar os logs em forma de index afim de analisar os logs de sua infraestrutura de maneira eficiente, direcionando assim todos os logs dos diversos ativos para um único host, facilitando uma auditoria e a identificação de algum problema.

Alternativamente, foram criadas esteiras no Azure Devops Service (serviço na nuvem), na organização CAIXA_AZURE, em: <https://dev.azure.com/CAIXAAZURE/>

Estas esteiras são restritas e seu uso está condicionado a autorização pela GEQTI/GESTI.

6. AZURE DEVOPS SERVER

Anteriormente conhecido como Team Foundation Server (TFS), o Azure DevOps Server é uma plataforma de ferramentas colaborativas de desenvolvimento de software que atuam dentro de um mesmo espaço de trabalho promovendo a automação da Entrega Contínua de Software.

O Azure DevOps Server integra-se facilmente a diversos IDE permitindo que o time trabalhe efetivamente no desenvolvimento de aplicações de diversos tamanhos.

Além dos módulos nativos, o ADS é expansível através da instalação de centenas de plugins.

6.1. COLEÇÕES E PROJETOS

A estrutura do espaço virtual de colaboração do ADS possui dois níveis: Coleção e Projeto.

O Projeto é um contêiner que representa um escopo de desenvolvimento e é composto por repositórios, pipelines, bibliotecas, grupos e times. Um projeto poderia ser compreendido como o espaço virtual de trabalho para criação e manutenção de sistemas, contendo todos os seus módulos e elementos apartados de outros “sistemas”.

Uma coleção é um agrupamento de projetos. Uma coleção representa um conjunto de projetos que possuem alguma afinidade ou interesse de agrupamento. Pode-se pensar numa Organização como uma coleção.

6.1.1. SINGLE PROJECT

É natural considerar que uma grande variedade de projetos que habitam uma coleção pode criar uma grande dificuldade em gerenciá-los. Os desafios comuns são:

- Adotar padrões de permissionamento a cada projeto devido à dificuldade de replicação ou herança em projetos distintos;
- lidar com uma visão fragmentada do trabalho de pessoas que atuam em diferentes frentes;
- planejar *verticalmente* o nível do portfólio (ou liderança) até o nível do projeto ou das equipes de recursos quando muitos fatores são transversais;
- planejar *horizontalmente* em vários projetos ou equipes de recursos, pois, inversamente ao item anterior, os recursos podem estar espalhados em diferentes projetos;
- Facilitar a mobilidade e a fluidez dos membros ou módulos da equipe entre projetos diferentes;
- Capturar Business Intelligence a partir de uma infinidade de projetos dispersos ou mesmo isolados;
- outras...

6.1.2. COLEÇÃO PROJETOS & PROJETO CAIXA

A CAIXA optou pelo uso do *single project* para os módulos dos sistemas que atendem as premissas do Modelo Corporativo e que possui compatibilidade com as tecnologias disponíveis.

Portanto, a coleção PROJETOS possui um projeto chamado projeto CAIXA.

As pipelines disponíveis no projeto CAIXA são as ideias para automação do processo de desenvolvimento e manutenção de aplicativos, com testes automatizados e publicação em ambientes de maneira contínua. Este projeto é totalmente suportado, escalável e aderente as melhores práticas, padrões e normas vigentes da CAIXA.

Por ser um single Projeto, o Projeto CAIXA precisou criar uma política de times e grupos complexa. O desafio de um único projeto onde todos os módulos e sistemas convivem no mesmo local é permitir o acesso a repositórios, pipelines e bibliotecas para aquele que precisam acessar, em um nível adequado, e negar o acesso a quem não pode ter.

A política de permissionamento está detalhada na seção “Permissionamento no ADS”.

Apesar de uma estrutura de permissionamento mais complexa, os desafios apresentados na seção SINGLE PROJECT foram em parte mitigados com essa solução. Caso a CAIXA opte por adotar o módulo AZURE BOARDS, os ganhos poderiam ser ainda maiores.

6.1.3. OUTRAS COLEÇÕES

Além da coleção PROJETOS, há uma coleção conhecida como LEGADO que visa oferecer os serviços de automação, publicação e testes automatizados para módulos ainda não compatíveis com o Modelo Corporativo de DevOps e, para módulos que precisam de simples rotinas de automação, há ainda a coleção de projetos BÁSICO, que não adota, a priori, templates e estruturas como o projeto CAIXA, sendo a simplicidade e o total controle das tarefas as vantagens para os desenvolvedores.



6.2. MÓDULOS & RECURSOS

O Azure Devops Server ou ADS é uma plataforma que oferece vários serviços aos usuários para que toda a cadeia de atividades/tarefas do processo de Entrega Contínua possa ser executada de forma automática, sustentável e replicável.

Portanto, os módulos representam os principais serviços ou recursos a disposição de cada área de projeto no ADS.

6.2.1. AZURE OVERVIEW

Oferece uma tela inicial de **overview**, ou seja, uma página para descrever o projeto com painéis para dar uma visão geral sobre a execução/evolução do projeto.

Além do overview, um **dashboard** para apresentação detalhada de painéis e consultas mais relevantes para o time. Aqui é possível acompanhar todo o conjunto de itens de trabalho que suportam as sprints de desenvolvimento.

Finalmente, uma wiki para documentação do projeto CAIXA e demais informações/registros que o time julgar pertinente.

6.2.2. AZURE BOARDS

Módulo para criação e gerenciamento dos itens de trabalho, painéis, consultas, planos, backlogs, sprints, etc.

Este módulo é orientado a gerenciar as atividades dos times de desenvolvimento e as organizações de trabalho em geral.

Atualmente, este módulo está inativo no projeto CAIXA.

6.2.3.AZURE REPOS

Embarcado no ADS, temos os repositórios de código fonte (e demais scripts) que irão nos apoiar para gerar toda a sorte de artefatos e controlar as versões das nossas mudanças.

Os repositórios utilizam a tecnologia GIT e disponibilizam toda a sorte de recursos que esta tecnologia permite, como políticas de branches, pull request, acrescido de outros recursos, como uma interface gráfica para gestão de commits, políticas de build e integração contínua.

6.2.4.AZURE PIPELINES

São organizações de rotinas que quando executadas consistem em uma série de tarefas automatizadas promovendo a disponibilização de uma nova versão de um software.

No ADS temos dois tipos de pipelines distintas: a Pipeline de Build, cujo objetivo é integrar e testar continuamente e construir artefatos para posterior publicação e, a Pipeline de Release, cujo objetivo é implantar continuamente em ambientes dinamicamente configurados e estrategicamente sequenciados para construir uma nova versão da aplicação plenamente funcional e disponível.

6.2.5.AZURE TEST PLANS

Módulo para criação de planos de teste automatizados e manuais nas esteiras. Para cada ciclo de desenvolvimento e publicação do produto, pode-se criar um plano de teste e/ou importar os casos de teste existentes para esse plano. Se desejar, pode-se dividir os casos de teste em conjuntos de testes separados dentro do plano para permitir o gerenciamento e o monitoramento mais fáceis desses conjuntos separados de casos de teste.

A disponibilidade deste módulo requer uma licença especial e está desativado no projeto CAIXA.

6.2.6.AZURE ARTIFACTS

Este Recurso realiza o gerenciamento de binários totalmente integrado a seus pipelines de CI/CD (integração contínua/implantação contínua). É possível compartilhar pacotes NuGet, Maven, Python e npm de fontes públicas e privadas para apoiar na compilação, gerenciamento e publicação.

Atualmente, este módulo está inativo no projeto CAIXA.

6.2.7.MARKETPLACE DE EXTENSÕES

Recurso para inclusão de extensões/plugins que vão estender o potencial da plataforma, desde extensões mantidas pela própria Microsoft, como muitos serviços criados pela comunidade.

6.3. PAPÉIS, TIMES & GRUPOS

Na opção de configuração do projeto CAIXA (project settings), há a opção de segurança (security), onde é possível realizar toda a gestão de ajuste de papéis, criação de grupos e permissionamentos. Na opção de times (team), é possível criar times e montá-los conforme os grupos criados.

6.3.1. PERMISSIONAMENTO NO PROJETO CAIXA

O Modelo Corporativo de DevOps adotou o *single project* para conter em um único projeto todas as esteiras de desenvolvimento.

A estrutura centralizada do *single project* onde todos os elementos estão acessíveis a todos, tornou-se um grande desafio e um modelo de permissionamento foi estruturado para refletir todos os papéis que a CAIXA possui na área de desenvolvimento, segmentá-los conforme seus mandatos de atuação, restringir acessos onde não são permitidos e montar times conforme as suas comunidades e áreas de atuação.

6.3.2. PRINCIPAIS PAPEIS NATIVOS

O ADS é, em sua essência, uma ferramenta colaborativa, portanto, uma ferramenta com uma política não centralizadora ou hierárquica.

Os perfis nativos da ferramenta são normalmente bem estabelecidos e autônomos conforme sua área de atuação.

O leitor ou reader, que possui acesso de leitura.

O colaborador ou contributor, que possui acesso de leitura, escrita e ações em repositórios e pipelines que um desenvolvedor normalmente faria.

O administrador do projeto ou project admin que pode criar, editar e excluir repos e módulos como um administrador normalmente faria.

Não há maiores granularidades de permissionamento entre estes principais papéis.

Porém, perfis nativos NÃO são utilizados no projeto CAIXA (projeto que reúne as esteiras corporativas), pois há muitas pipelines que automatizam módulos de sistemas críticos, muitos terceirizados envolvidos e perfis como contribuidor e administradores que podem editar e excluir elementos não é compatível com uma governança que nos exige tratar riscos em todas as suas dimensões e níveis.

Neste caso, foi necessário criar perfis “controlados” para não permitir, por exemplo, que um contibutor possa excluir uma branch, repo ou editar uma pipeline, uma vez que está habilitada esta permissão por padrão de ferramenta.

Seguem abaixo os perfis criados exclusivamente para o projeto CAIXA:

6.3.3. DESENVOLVEDOR

Baseado no perfil nativo de contributor, o perfil de desenvolvedor e o usuário responsável pelo desenvolvimento e/ou manutenção de versões de código dos softwares e a automações de seu desenvolvimento, teste e publicação em ambientes.

Esse perfil pode submeter códigos e scripts, criar branches, commits, solicitar Pull Requests, entre outras atividades.

6.3.3.1. DIFERENÇAS ENTRE O PAPEL CONTRIBUTOR (NATIVO) E O PAPEL DE DESENVOLVEDOR (CUSTOMIZADO)

Abaixo seguem as diferenças entre o perfil contributor (esquerda) e o perfil customizado desenvolvedor (direita) no uso dos repositórios.

ACCESS CONTROL SUMMARY

Shows information about the permissions being granted to this identity

Bypass policies when completing pull requests	Not set
Bypass policies when pushing	Not set
Contribute	Allow
Contribute to pull requests	Allow
Create branch	Allow
Create repository	Not set
Create tag	Allow
Delete repository	Not set
Edit policies	Not set
Force push (rewrite history, delete branches and tags)	Not set
Manage notes	Allow
Manage permissions	Not set
Read	Allow
Remove others' locks	Not set
Rename repository	Not set

ACCESS CONTROL SUMMARY

Shows information about the permissions being granted to this identity

Bypass policies when completing pull requests	Deny
Bypass policies when pushing	Not set
Contribute	Allow
Contribute to pull requests	Allow
Create branch	Allow
Create repository	Not set
Create tag	Allow
Delete repository	Not set
Edit policies	Not set
Force push (rewrite history, delete branches and tags)	Not set
Manage notes	Allow
Manage permissions	Not set
Read	Not set
Remove others' locks	Not set
Rename repository	Not set

Há muitas semelhanças e duas diferenças significativas entre esses dois papéis:

Enquanto o contributor PODE herdar permissões que permitem SOBREPOR políticas de branches (not set), no perfil desenvolvedor esta política é EXPLICITAMENTE negada (deny).

No projeto CAIXA, as políticas de branches, uma vez definidas, não devem permitir ao desenvolvedor, CAIXA ou FSW, deixar de atendê-las ou alterá-las, salvo exceções.

Outra diferença é quanto a opção de leitura. Nativamente, o contributor é “reader allow”, ou seja, lê todos os repos do mesmo projeto, mas só escreve naqueles que tem a permissão como contribuidor. Isso faz sentido quando há diversos projetos diferentes e cada portfólio está contido no seu próprio projeto.

Mas, no projeto CAIXA, onde a opção é de single Project, todos os repositórios estão disponíveis em um mesmo projeto, até módulos críticos. Neste caso, a opção de leitura a todos os repositórios é inviável para qualquer desenvolvedor. Cada time deve ver apenas aos repositórios que tem acesso. Portanto, a opção “reader allow” é a

opção para “chavear” o acesso ao repositório, que inicialmente está “not set” ou invisível” ao perfil desenvolvedor.

É importante ressaltar que a opção “read allow” no projeto CAIXA é atribuída **ao time em que o desenvolvedor faz parte**, e nunca somente ao desenvolvedor ou diretamente a matrícula. A opção de “read” para o time garante que todos os papéis do time possam atuar conforme o perfil.

O perfil também terá acesso a executar Builds Pipelines e Releases Pipeline para publicação em ambientes de DEV e TQS.

6.3.4. QUALIDADE

Identifica-se como perfil de Qualidade aqueles que são responsáveis por atuar como especialistas em Controle de Qualidade.

Compete a este perfil garantir a realização das análises estática de código, segurança, code review e/nas aprovações de pull request.

Este é um perfil restrito e exclusivo para funcionários CAIXA.

6.3.5. SUPORTE (ANTIGO SAI)

Identifica-se como perfil de Suporte o usuário com maior abrangência de criação de estrutura de uso, podendo criar e fazer manutenção em várias etapas do processo.

Esse perfil é o responsável por criar e manter grupos locais e times e seus padrões, sempre atendendo as solicitações formais a ele designadas. Ele também é responsável pela manutenção de repositórios, pipelines, stages e bibliotecas.

Este perfil é exclusivo da CETAD e GECEQ

6.3.6. SEGURANÇA

Perfil responsável pelo permissionamento dos papéis de operações (aprovadores, implantação e produção) nos stages de HMP e PRD, além dos grupos de variáveis associados.

Este perfil é acionado exclusivamente pelo pessoal de operações e deverá garantir a estes todo o permissionamento necessário, além de vedar o acesso a perfis não autorizados.

6.3.7. APROVADOR

Perfil responsável por pré-aprovar ou pós-aprovar os stages de uma pipeline de release.

Os aprovadores dos stages de HMP e PRD são exclusivos para as áreas de operações. Por padrão, todos os stages de HMP e PRD depende de aprovadores.

6.3.8. OPERAÇÃO

Perfil responsável por acompanhar os releases das aplicações nos ambientes de HMP ou PRD e manter o pleno funcionamento e monitoramento destes ambientes.

6.4. GRUPOS & TIMES

6.4.1. GRUPOS DE COMUNIDADE NO ADS

Após conhecer os diferentes tipos de perfis, é necessário agrupar os membros de um perfil em um grupo para que possa atuar dentro de uma comunidade ou unidade de desenvolvimento. Entende-se como grupo no ADS, o agrupamento de membros que possuem o mesmo perfil.

Numa comunidade podem existir grupos de desenvolvedores CAIXA, desenvolvedores de fábrica e qualidade. Além destes grupos, existem os grupos que são transversais as comunidades, como Suporte, Aprovadores, Operadores e Segurança.

Os grupos no ADS são criados sempre que a Comunidade solicitar a inclusão/migração de um sistema para o devops.caixa ou a qualquer momento, quando surgir um grupo novo na comunidade, como uma nova fábrica.

A nomenclatura padrão dos Grupos no ADS é:

XPTO-dev,

onde XPTO é o nome da comunidade (em letras maiúsculas e o perfil vem após o hífen em letras minúsculas.

Outros exemplos:

XPTO-qa, onde qa = qualidade

XPTO-dev-fswa, onde fswa = nome qualquer de fábrica.

6.4.2.GRUPOS NO AD (ACTIVE DIRECTORY)

Sempre haverá uma associação de um grupo no ADS com um grupo no AD. É no grupo de AD que membros são incluídos e excluídos, portanto, sempre que uma pessoa quiser ser incluída no ADS para fazer parte de um determinado grupo numa comunidade ele precisa ser incluído no grupo de AD associado.

Quem gerencia os grupos de AD da comunidade é a PRÓPRIA comunidade (gestores).

A nomenclatura padrão dos Grupos no ADS é:

G CC0000 dev,

onde G CC0000 é o grupo de uma centralizadora, como G DF7390 e dev é o perfil associado.

Por exemplo: A Comunidade XPTO precisa ter um grupo de desenvolvedores da Fábrica "A" para atuar numa parte do seu portfólio. Ela informa na solicitação (ver seção de suporte como solicitar serviços) que seja criado o grupo de desenvolvedores da fábrica "A" da Comunidade. O Suporte criará o grupo ADS: XPTO-decv-fswa e o vinculará ao grupo de AD respectivo, como "G CC9999 dev-fswa.

6.4.3.REUSO DOS GRUPOS DE AD NO CLEARCASE

Os grupos de AD de desenvolvedores CAIXA ou FSW são os mesmos utilizados para acesso ao ClearCase, isso garante acesso automático de um usuário cadastrado em qualquer uma das duas ferramentas.

6.4.4.GRUPOS TRANSVERSAIS

Os grupos transversais são grupos mantidos diretamente pela CETAD ou GECEQ (administração da ferramenta). São grupos estáveis e seus membros são gerenciados pelas próprias unidades, como CEPTI (aprovadores, implantadores e operações), similar ao modelo de gestão dos grupos locais.

6.4.5.TIMES

Times são arranjos de grupos numa comunidade.

Se uma comunidade só possui desenvolvedores CAIXA, provavelmente precisaria de apenas um time e este time seria composto pelos seguintes grupos:

- XPTO-dev (desenvolvedores CAIXA da comunidade)
- XPTO-qa (agentes de qualidade da comunidade)
- Suporte (suporte transversal)
- Operação (operação transversal)
- Segurança (segurança transversal)

Porém, este não é a realidade da maioria das comunidades. As comunidades mantêm muitos itens de portfólio (sistemas, projetos, manutenções...) e trabalha com muitos prestadores em fábricas de software distintas. Portanto, muitos dos prestadores que apoiam a comunidade não podem acessar boa parte do portfólio da comunidade (por serem contratos distintos e não possuírem termo de confidencialidade). Assim sendo, é necessário realizar várias combinações de grupos conforme a diversidade do portfólio.

Supondo que além de um portfólio que seja exclusivo de desenvolvedores CAIXA, a comunidade XPTO possua contrato de suporte com dois outros contratos da fábrica: A e B. Neste caso, são necessários mais dois times a serem agregados:

Time da FSWA

- XPTO-dev (desenvolvedores CAIXA da comunidade)
- XPTO-dev-fswa (desenvolvedores da fábrica A da comunidade)
- XPTO-qa (agentes de qualidade da comunidade)
- Suporte (suporte transversal)
- Operação (operação transversal)
- Segurança (segurança transversal)

Time da FSWB

- XPTO-dev (desenvolvedores CAIXA da comunidade)
- XPTO-dev-fswb (desenvolvedores da fábrica B da comunidade)
- XPTO-qa (agentes de qualidade da comunidade)
- Suporte (suporte transversal)
- Operação (operação transversal)
- Segurança (segurança transversal)

A necessidade de arranjos de novos times na comunidade é proporcional a diversidade de fábricas no seu portfólio.

Caso haja necessidade da criação de um novo grupo ou time na comunidade, deve-se solicitar este serviço conforme a seção SUPORTE.

6.5. PROJETOS NO ADS

Uma coleção é um coletivo de projetos e cada coleção representa uma organização específica de projetos afins. Cada projeto possui uma estrutura distinta de boards, repos, pipelines, task groups, libraries e papéis.

6.5.1. CRIAR PROJETOS NAS NOVAS COLEÇÕES

O perfil de Administrador de Coleção (CETAT / GECEQ) pode criar/manter projetos, definido o nome, descrição, a tecnologia para controle de versão e o tipo de estrutura para os itens de trabalho, do mais básico ao mais formal (CMMI).

The screenshot shows a 'Create new project' dialog box with a dark theme. It includes a close button (X) in the top right corner. The form contains the following fields and options:

- Collection name:** A text field with the value 'DefaultCollection'.
- Project name:** A text field with a red asterisk indicating it is required.
- Description:** A larger text area for project details.
- Advanced:** A section header with a chevron icon.
- Version control:** A dropdown menu currently set to 'Git'.
- Work item process:** A dropdown menu currently set to 'Agile', which is expanded to show a list of options:
 - Agile (selected)
 - Custom Agile
 - Basic
 - Custom Basic
 - CMMI
 - CMMI_CAIXA
 - CMMI_Sync
 - Scrum

Projeto criado, no Project Settings, opção overview, é possível habilitar os serviços que estão disponíveis, ajustar detalhes ou mesmo excluir o projeto.

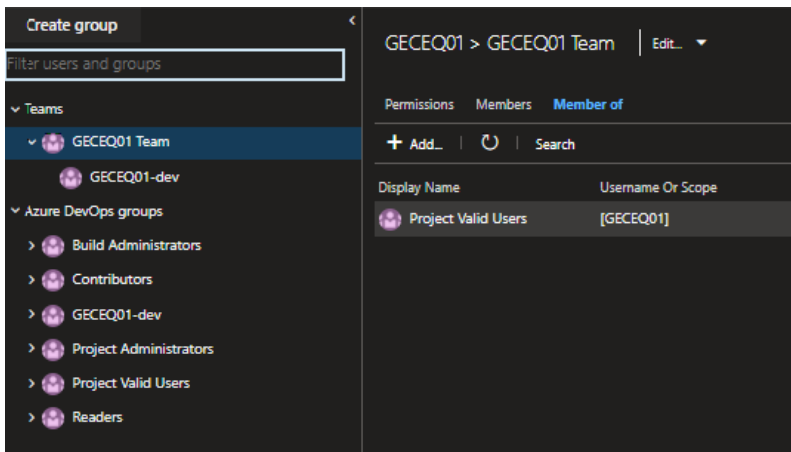
The screenshot shows the Azure DevOps interface for the 'GECEQ01' project. The left sidebar contains a navigation menu with options: Overview, Boards, Repos, Pipelines, Test Plans, and Artifacts. The main content area is titled 'Project details' and includes a 'Name' field with the value 'GECEQ01', a 'Description' field with the text 'area de teste para a geceq01', and a 'Save' button. Below this, the 'Azure DevOps services' section lists several services with toggle switches: Boards (On), Repos (On), Pipelines (On), Test Plans (On), and Artifacts (On). At the bottom, there is a 'Delete project' section with a warning message and a 'Delete' button.

6.5.2.GRUPOS E PERFIS NATIVOS

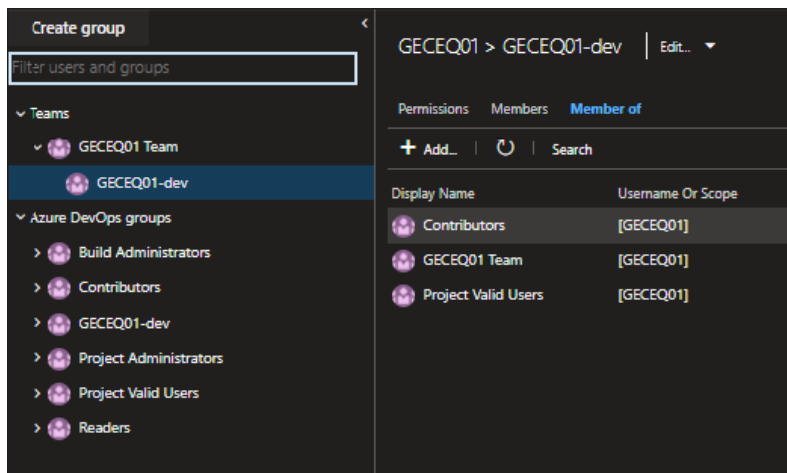
Na opção settings, qualquer administrador do projeto pode criar e organizar os grupos e times conforme seus interesses, desde que adotes os padrões estabelecidos neste documento para a sua coleção.

Como grupos e times pertencem ao projeto, não é possível “puxar” grupos de outros projetos (uma vez que são nos grupos que o mapa de permissionamento acontece).

É recomendável que os times criados sejam vinculados apenas ao Project Valid Users.



Os grupos de desenvolvedores do time devem estar vinculados ao perfil Contributors.



O perfil Contributor permite realizar todas as atividades necessárias para um desenvolvedor.

GECEQ01 > GECEQ01-dev | Edit...

PermissionsMembersMember of

Bypass rules on work item updates

Not set

Change process of team project.

Not set

Create tag definition

Allow (inherited) Why?

Create test runs

Allow (inherited)

Delete and restore work items

Allow (inherited)

Delete shared Analytics views

Not set

Delete team project

Not set

Delete test runs

Allow (inherited)

Edit project-level information

Not set

Edit shared Analytics views

Not set

Manage project properties

Not set

Manage test configurations

Allow (inherited)

Manage test environments

Allow (inherited)

Move work items out of this project

Not set

Permanently delete work items

Not set

Rename team project

Not set

Suppress notifications for work item updates

Not set

View analytics

Allow (inherited)

View project-level information

Allow (inherited)

View test runs

Allow (inherited)

Clear explicit permissions

Save changes

Undo changes

Trace permission

GECEQ01

TRACE INHERITANCE RESULTS

Permission	Create tag definition
Effective value	Allow (inherited)
Identity	[GECEQ01]GECEQ01-dev

GROUP MEMBER INHERITANCE

The permission value is being inherited through your direct or indirect membership in these groups

[GECEQ01]Contributors Allow

Security for GECEQ01 repository

SecurityOptions

+ Add...

Inheritance

Search

Azure DevOps groups

Project Collection Administrators

Project Collection Build Service Accounts

Project Collection Service Accounts

Build Administrators

Contributors

Project Administrators

Readers

Users

Manuel Nunes Pereira

ACCESS CONTROL SUMMARY

Shows information about the permissions being granted to this identity

Bypass policies when completing pull requests	Not set
Bypass policies when pushing	Not set
Contribute	Allow (inherited)
Contribute to pull requests	Allow (inherited)
Create branch	Allow (inherited)
Create tag	Allow (inherited)
Delete repository	Not set
Edit policies	Not set
Force push (rewrite history, delete branches and tags)	Not set
Manage notes	Allow (inherited)
Manage permissions	Not set
Read	Allow (inherited)
Remove others' locks	Not set
Rename repository	Not set

Clear explicit permissions

Remove

Save changes

Undo changes

Permissions for Projeto XPTO

+ Add...

Search

Project Collection Administrators

Project Collection Build Administrators

Project Collection Build Service Accounts

Project Collection Test Service Accounts

Build Administrators

Contributors

Project Administrators

Readers

ACCESS CONTROL SUMMARY

Shows information about the permissions being granted to this identity

Administer build permissions	Not set
Delete build pipeline	Allow
Delete builds	Allow
Destroy builds	Allow
Edit build pipeline	Allow
Edit build quality	Allow
Manage build qualities	Not set
Manage build queue	Not set
Override check-in validation by build	Not set
Queue builds	Allow
Retain indefinitely	Allow
Stop builds	Allow
Update build information	Allow
View build pipeline	Allow
View builds	Allow

Clear explicit permissions

Remove

Save changes

Undo changes

47

6.6. REPOSITÓRIOS NO ADS

Sistemas de controle de versão são softwares que ajudam a controlar as alterações feitas em seu código ao longo do tempo. Ao editar seu código, você pode gerar snapshots ou instantâneos dos arquivos que você está desenvolvendo e eles serão mantidos indefinidamente na sua pasta local. Além disso, você pode atuar em branches ou ramos específicos de desenvolvimento que permite paralelismos e trabalhos em equipe, quando idealmente gerenciado.

6.6.1.PADRÕES DE NOMENCLATURA

Os padrões de nomenclatura a serem adotados é o nome do sistema, com cinco caracteres maiúsculos, seguido de um caractere “hífen” e um nome que identifica o módulo deste sistema a qual este repositório representa em caracteres minúsculos.

Por exemplo, o sistema “ABC” é composto de vários módulos e, consequentemente, vários repositórios, cada qual com um escopo de compilação. Um dos repositórios contém o código para uma solução para um módulo de front-end.

Neste caso, o repositório deverá ser identificado da seguinte forma: SIABC-frontend (sem espaço).

6.6.2.TIPOS DE REPOSITÓRIOS

Os repositórios no Azure DevOps devem ser criados dentro do contexto do módulo, sendo este um insumo para o processo de geração do artefato (pipeline de build) e para manter os códigos de configuração para o processo de publicação/manutenção dos ambientes (pipeline de release).

Os repositórios de configuração precisam adicionar mais um termo na nomenclatura e o termo é -config.

Neste caso, o módulo de front-end do sistema SIABC terá dois repositórios:

SIABC-frontend

SIABC-frontend-config

6.6.3.CRIAÇÃO DE REPOSITÓRIOS

A criação de um repositório é realizada apenas pelo perfil SUPORTE.

A criação pode ser simplesmente utilizando a interface de criação no ADS ou migrando de um outro repositório remoto utilizando uma URL válida e com rota garantida (acesso de rede).

Deve-se adicionar as configurações de SSL (certificado) para utilização do git. Esta orientação e o certificado estão disponíveis na wiki do projeto CAIXA (erros conhecidos -> Git - Resolução do erro SSL)

Quando não for possível realizar a migração, deve-se clonar de forma espelhada o repositório;

É importante reforçar a premissa de que os repositórios devem ser criados para módulos do sistema, estes quando independentes e versionáveis, por exemplo, frontend e backend;

Deve-se definir como branch padrão, a branch de integração ou develop (Set as default branch);

Para usuários do nodejs, deve-se criar a pasta. s2i/bin e os arquivos "run", "assemble". (pode-se encontrar os modelos dos arquivos no repositório "Sample-Node")

Abrir o arquivo "run" e verifique se ele está apontando para o "app.js" corretamente (Wiki)

Nos repositórios de configuração, deve-se ajustar os arquivos conforme respectivas tecnologias. Ajustar ENVIRONMENTS (Variables Group) nos demais arquivos de configurações conforme linguagens para o processo de build:

Angular e node: environments/*

Java: jmx_prometheus.yaml

6.6.4.ESPELHANDO REPOSITÓRIOS

O processo de espelhamento de repositórios ...

- Abra Git Bash.
- Crie um clone bare espelhado do repositório.

```
$ git clone --mirror  
https://github.com/exampleuser/repository-to-mirror.git
```

- Defina o local de push no espelho.

```
$ cd repository-to-mirror  
$ git remote set-url --push origin  
https://github.com/exampleuser/mirrored
```

Assim como um clone bare, um clone espelhado inclui todos os branches remotes e tags, mas todas as referências locais serão substituídas todas as vezes que você fizer fetch, assim ele sempre será o mesmo do repositório original. O push no espelho é simplificado pela configuração da URL para pushes.

- Para atualizar o espelho, obtenha atualizações e faça push.

```
$ git fetch -p origin  
$ git push --mirror
```

6.6.5.PERMISSIONAMENTO PARA OS REPOSITÓRIOS

Repositório criados, é importante permissioná-lo para o time correto (perfil SUPORTE).

Um repositório contém códigos de um módulo de um sistema, que por sua vez, pertence ao portfólio de uma Comunidade (XPTO) e, normalmente, suportado por uma fábrica de software (FSWA).

Portanto, o repositório deve ser permissionado para o XPTO-dev-fswa habilitando a opção READ para o time:

ACCESS CONTROL SUMMARY	
Shows information about the permissions being granted to this identity	
Bypass policies when completing pull requests	Not set
Bypass policies when pushing	Not set
Contribute	Not set
Contribute to pull requests	Not set
Create branch	Not set
Create tag	Not set
Delete repository	Not set
Edit policies	Not set
Force push (rewrite history, delete branches and tags)	Not set
Manage notes	Not set
Manage permissions	Not set
Read	Allow
Remove others' locks	Not set
Rename repository	Not set

Se a opção READ não estiver habilitada (allow), então o time não poderá ver e atuar no repositório conforme o papel.

Portanto, para se acessar um repositório é necessário que um funcionário CAIXA ou terceirado da Comunidade:

- 1) ser incluído pelo gestor da comunidade para ser membro do grupo de AD correspondente, e
- 2) o time ter permissão de leitura no repositório habilitado.

Caso o funcionário foi adequadamente incluído pelo Gestor da Comunidade no grupo de AD e seu acesso ainda não estiver funcionando, deve-se abrir um serviço para o SUPORTE (ver seção SUPORTE).

6.6.6.RESTRINGIR ACESSO AOS ANTIGOS REPOSITÓRIOS

Assim que os repositórios são migrados e o time “virar a chave”, os antigos repositórios devem ter seus acessos restritos, para evitar algum tipo de conflito entre repositórios remotos.

No caso do fontes.caixa, os projetos migrados devem ser EXCLUÍDOS da base, seja para evitar qualquer atualização indevida, seja por ser uma ação de expurgo. Uma vez que se trata de repositórios git, a migração em si já é uma cópia segura, além dos códigos que provavelmente ainda estão nas máquinas locais por ser o último commit.

6.6.7. TIPOS DE ARQUIVOS

Os arquivos a serem mantidos nos repositórios do Azure DevOps devem ser exclusivamente do tipo “plain text” ou texto plano.

Estes tipos de arquivos são mais facilmente gerenciados pela tecnologia Git, permitindo um controle de versão mais efetivo e pleno uso do recurso “diff” para revisão de código.

Arquivos de configuração de ambiente (como standalone.xml), scripts de ambiente, variáveis de ambiente, devem ser versionados no repositório de configuração.

Excepcionalmente, pequenos arquivos binários que fazem parte do escopo da versão construída (compilada ou empacotada), como arquivos de imagem em uma criação de página WEB ou outros formatos, podem ser mantidos no repositório Git de fontes.

Arquivos de documentação devem ser mantidos em repositórios adequados e não mantidos no git, exceto pequenos arquivos texto, como README, em formatos txt ou md.

Arquivos compilados, backups, imagens, compactados e demais arquivos binários **não devem ser mantidos nos repositórios git.**

A Branch de integração pode assumir nomes como develop ou desenv, sem prejuízo para a sua finalidade e/ou utilidade.

O merge com a Branch de integração deve ser realizado preferencialmente via Pull Request para a oportunidade de solução de conflitos das versões candidatas, revisão de código por um especialista e/ou integração conforme estratégias de desenvolvimento.

6.6.9.PULL REQUEST

O Pull Request é uma maneira do desenvolvedor sinalizar a conclusão de uma versão da atualização que estava sob sua responsabilidade. Ao finalizar a sua atividade, o desenvolvedor realiza a sua entrega através de uma solicitação de Pull Request.

Através do Pull Request, os demais colaboradores sabem que é o momento de entrega e que é necessário realizar o merge do código na Branch de integração.

Exemplos de situações em que o Pull Request pode ser solicitado:

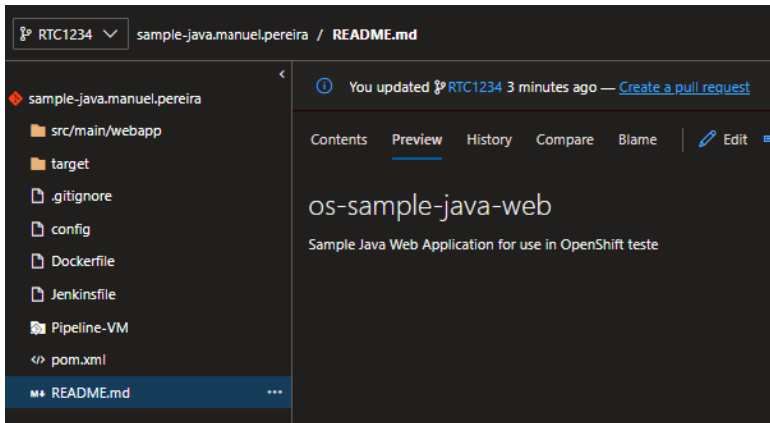
Pela Fábrica, na entrega do seu código, preferencialmente previamente testado e compilado, para iniciar o processo de compilação automatizado pela CAIXA (que será testado automaticamente com as suas rotinas de verificação).

Pelo time de desenvolvimento interno (quando internalizado), para iniciar o processo de automação.

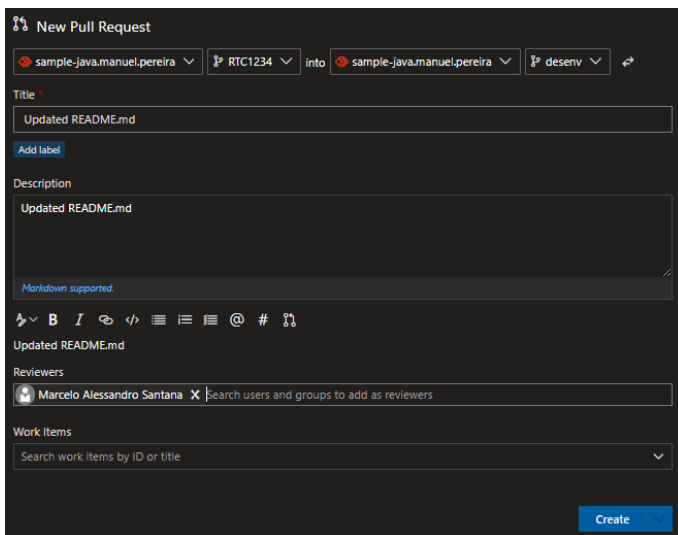
O uso do recurso Pull Request ainda que não obrigatório, é altamente recomendado, especialmente em equipes com vários desenvolvedores, trabalhos paralelos e sistemas críticos.

6.6.9.1. COMO CRIAR UM PULL REQUEST

As alterações realizadas no código sempre são passíveis de solicitação de pull request e a opção de criação de pull request são facilmente localizadas na interface do ADS nos repos correspondentes.



É recomendado que sempre que houver necessidade de merge na branch de integração, que o mesmo ocorra por solicitação de pull request.

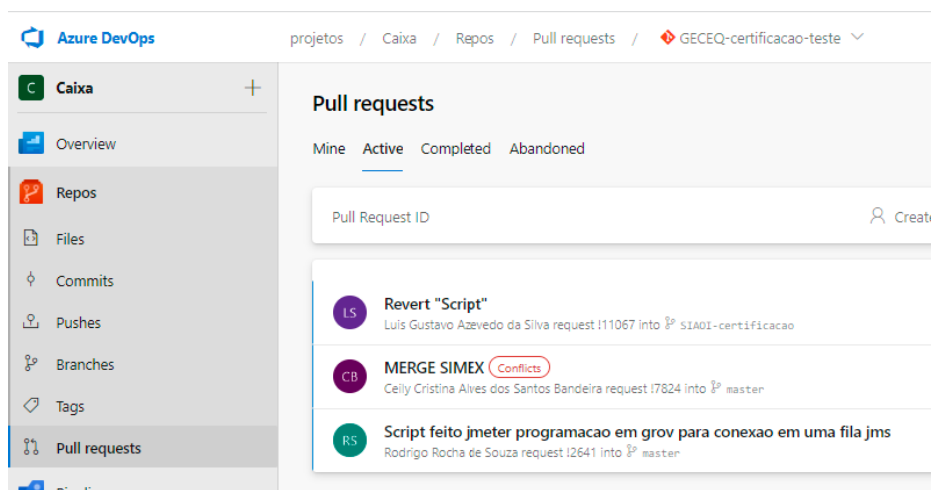


Deve ser incluído um aprovador distinto do próprio desenvolvedor, ainda que não obrigatório, para um caráter de integração e revisão de qualidade.

Caso haja políticas habilitadas na branch de destino, como permissionamento de contribuição em branches protegidas, o aprovador deverá ser necessariamente alguém com o perfil de Qualidade.

6.6.9.2. COMO APROVAR UM PULL REQUEST

O aprovador deverá selecionar o repositório de atuação e verificar na opção “pull request” se há solicitações ativas para ele.

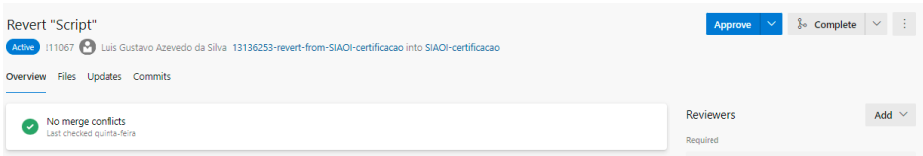


The screenshot displays the Azure DevOps web interface. The top navigation bar shows the path: projetos / Caixa / Repos / Pull requests. The left sidebar contains a menu with options: Overview, Repos, Files, Commits, Pushes, Branches, Tags, and Pull requests (which is highlighted). The main content area is titled 'Pull requests' and includes tabs for Mine, Active, Completed, and Abandoned. Below these tabs is a search bar labeled 'Pull Request ID' and a 'Create' button. A list of pull requests is shown, each with a user icon, a title, and details about the request and target branch:

- Revert "Script"** by Luis Gustavo Azevedo da Silva, request 111067 into SIAOI-certificacao
- MERGE SIMEX** by Celly Cristina Alves dos Santos Bandeira, request 17824 into master. This request has a red 'Conflicts' label.
- Script feito jmeter programacao em grov para conexao em uma fila jms** by Rodrigo Rocha de Souza, request 12641 into master

Após identificar as solicitações que dependem da sua aprovação, deve selecionar a solicitação e, caso pertinente e sem conflitos, aprovar o pull request e completar o merge na branch destino.

É importante verificar se o aprovador tem ou não permissão de contribuição na branch de destino. Se o aprovador não tiver, o merge não será efetivo.



Após a atualização da branch de destino e, caso haja um trigger de início automático da pipeline de build, o processo de integração contínua será automático para publicação do artefato no binarios.caixa.

6.6.10. CONTROLE SEMÂNTICO DE VERSÃO

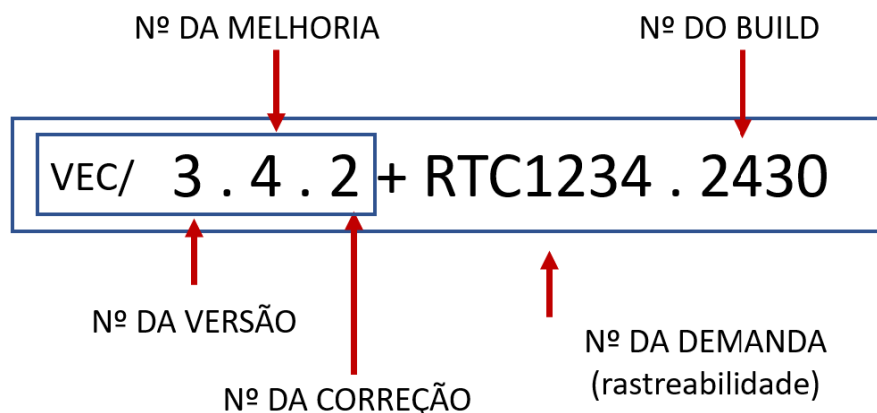
O Git possui um poderoso recurso que é o de criar marcação ou tags em momentos importantes do processo de criação e/ou manutenção dos aplicativos, como os commits de entrega ou releases.

É importante que as tags tenham valor semântico, ou seja, possuam significado para os times de desenvolvimento e demais interessados identificar as versões em que trabalham.

Uma tag deve ser capaz de identificar se o commit representa um simples snapshot, uma versão candidata ou release candidate (RC), se o desenvolvimento é uma grande mudança do sistema, uma simples melhoria, uma correção, qual a nova versão do sistema ou qualquer outro tipo de informação que possa ajudar o time a garantir um bom controle de versão e sua rastreabilidade.

Portanto, a cada marco importante do desenvolvimento, a marcação semântica deve ser complementar ao processo de controle de versão adotados nos commits e branches.

6.6.11. PADRÃO PARA TAGS



O padrão de formação a ser adotado na tag deve considerar quatro casas, separadas por ponto, sem prejuízo de outras adoções com mais casas, desde que as casas iniciais sigam o padrão nesta ordem:

Major . Minor . Patch . Build

Major: quando a mudança é tão significativa que afeta a compatibilidade com a versão anterior.

Minor: quando é adicionada uma melhoria com novas funcionalidades, sem mudanças significativas na aplicação.

Patch: correção de bug em produção (hotfix).

Build: indica o número de builds ou compilações executadas pelas rotinas de Integração.

Exemplo: tag “2.15.3.3434”, onde:

2 – Versão do sistema / aplicativo plenamente compatível

15 - Última melhoria implementada
3 – última correção implementada.
3434 - Último build / compilação gerada

Na publicação de uma nova correção:

Nova versão: 2.15.4.3440

Na publicação de uma nova melhoria:

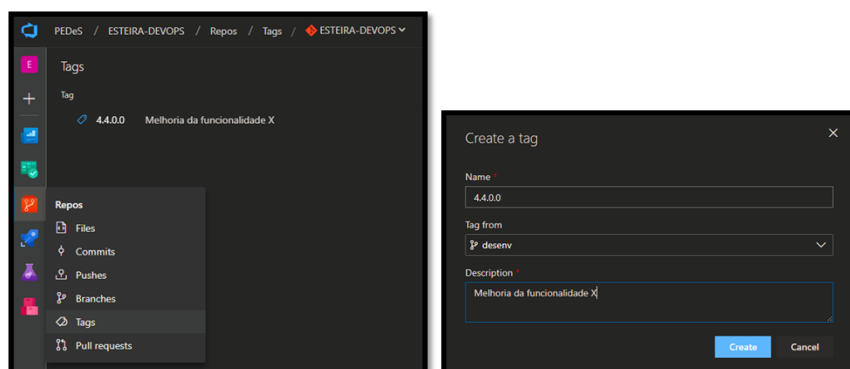
Nova versão: 2.16.0.3440

Na publicação de uma nova versão:

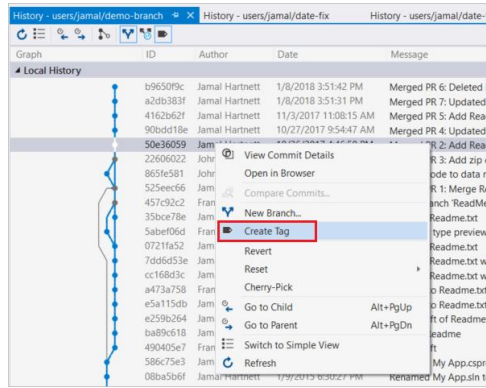
Nova versão: 3.0.0.3440

6.6.12. CRIAÇÃO DAS TAGS NO ADS

Tags podem ser criadas por vários motivos e, manualmente, a criação de uma tag é através da opção Repos, no menu do projeto.



Uma tag também pode ser criada diretamente na exibição do histórico de commits (tela de commits), basta clicar com o botão direito do mouse na confirmação desejada e escolher a opção: Criar tag.



6.6.13. CONTROLE SEMÂNTICO DAS PIPELINES

Na execução das pipelines, deve-se incluir o número de versão aderente ao padrão semântico aqui detalhado.

Assim, a pipeline irá apresentar visualmente aos interessados em acompanhar a sua execução, um nome que tenha valor semântico para o time, que identifique qual a versão candidata que o time está desenvolvendo/automatizando neste momento.

Variables		
system.debug		false
version.app		5.3.2

Este valor incluído no nome da pipeline é manual e não tem vinculação direta com uma tag criada no repositório git. A vinculação é apenas semântica, ou seja, é o time que deve ter o cuidado para inserir valores consistentes e convergentes e manualmente controlar a correta grafia para ambos nos momentos adequados, uma vez que o ADS não faz isso automaticamente.

Devido a esta ausência de sincronização automática entre os módulos de pipelines e repositórios no ADS, foi criada uma rotina automática para controle de versão.

6.6.14. PERMISSIONAMENTO PARA BRANCHES

Não há nenhum permissionamento criado automaticamente para branches, portanto, o permissionamento padrão do repositório (para os perfis de dev e qa) são herdados para todas as branches do repositório.

Para limitar o acesso a branches como a master e a develop dos desenvolvedores (para quem adota o modelo do gitflow), recomenda-se solicitar este bloqueio para a unidade responsável por definir políticas de GCM.

O procedimento é simples: incluir a opção de “deny” para a ação de “contribute” na branch que se quer limitar (ver imagem).

Para o perfil de qualidade, a ação de contribuição deve ser mantida, pois é um perfil exclusivo para funcionários CAIXA e recomendada para os agentes de qualidade/integradores do time.

develop

☒ Inheritance ⓘ

Search for users or groups

▼ Azure DevOps Groups

-  .Desenvolvedor
-  .Operacoes
-  .Qualidade
-  .SAI
-  .Seguranca

[Caixa]\.Desenvolvedor

Bypass policies when completing pull requests

Bypass policies when pushing

Contribute

Edit policies

Force push (rewrite history, delete branches and tags)

Manage permissions

Remove others' locks

Deny (inherited) ▼ ⓘ

Not set ▼

Deny ▼ ✓

Not set ▼

Not set ▼

Not set ▼

Not set ▼

6.7. ESTEIRAS DE DEVOPS

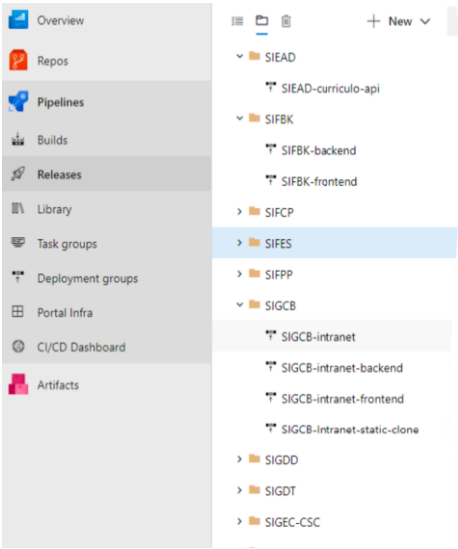
Há dois tipos de pipelines disponíveis no ADS:

- (I) pipelines de build (ou simplesmente pipelines), que geram artefatos ou pacotes e são publicado no nexus (binarios.caixa)
- (II) pipelines de release, compostos por stages, que geram snapshots ou releases para publicação nos ambientes.

Como o modelo de *single project* nos leva a um cenário de concentração de todas as pipelines disponíveis num único projeto, o projeto CAIXA adotou uma estrutura de pastas para facilitar a localização das pipelines.

Cada pasta tem o nome de um sistema e as pipelines ou módulos que pertençam a este sistema devem ser incluídos na pasta. Uma pasta pode ter várias pipelines, como back-end, front-end, entre outras, conforme estratégia do time para a criação de novas versões dos módulos que compõe o seu sistema.

O permissionamento de acesso/escrita das pipelines é na pasta e para um único time, o que significa que basta estar numa pasta que a pipeline herda o seu permissionamento.



6.7.1. PIPELINES DE BUILD

As pipelines de build são automatizadoras de tarefas que usam como principal insumo as novas versões dos códigos fontes geradas, e que realizam tarefas de preparação, verificação de qualidade, geração de pacotes e publicação em um repositório adequado.

Pipelines de build habilitam o processo de integração contínua, onde novas versões dos códigos podem ser continuamente integradas e compiladas.

É possível que uma esteira quebre durante as suas tarefas, seja por erro de compilação, seja porque a qualidade do código está abaixo dos parâmetros previstos ou por existência de muitas vulnerabilidades. Neste caso é necessário promover todos os ajustes necessários para obter o resultado esperado que é o sucesso na execução da pipeline e a publicação do pacote no Nexus (binarios.caixa).

Por padrão, os builds ficam disponíveis no *workspace* do ADS pelo período de 30 dias para ser utilizado pela pipeline de release. Após esse período, é necessário rodar uma nova pipeline de build (pode apontar para o mesmo commit) e gerar um novo artefato.

É possível gerar uma marcação para reter o pacote (símbolo de cadeado) e evitar esta política de expurgo, quando for o caso.

6.7.1.1. SNAPSHOT

O repositório de binários não permite que você envie uma mesma versão de artefato por uma questão de integridade com as versões candidatas.

Mas o que fazer quando se está compilando versões de teste ou versões que não são consideradas como RC (release candidate)?

Neste caso, deve-se utilizar o sufixo -SNAPSHOT, após a informação do controle semântico de versão na interface de execução da pipeline (version.app)



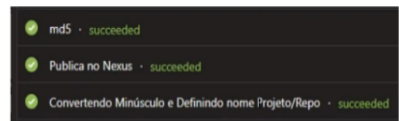
A utilização deste termo evitará a quebra da pipeline por violação de integridade e permitirá a publicação desejada.

6.7.1.2. ARTEFATOS COMO PRODUTOS

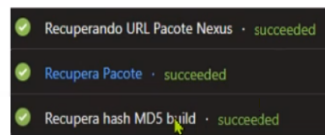
Os artefatos que são produtos gerados pela pipeline de build se tornam insumo para recuperação futura na pipeline de release, inclusive com verificação de hash para garantir a integridade.



O pacote é enviado durante o Build...



E recuperado durante o Release

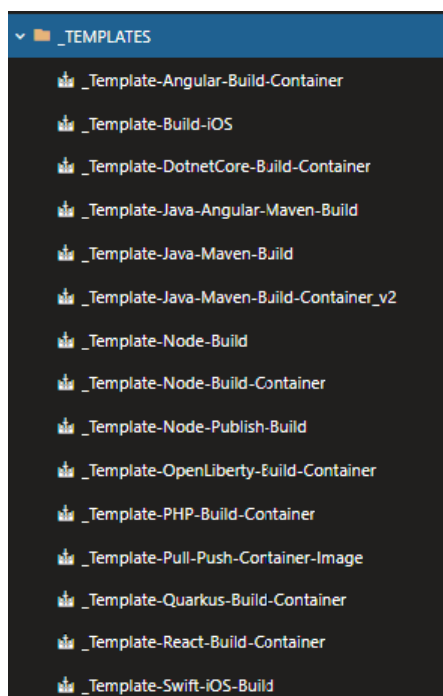


6.7.1.3. TEMPLATES

As pipelines de build não são construídas de forma ad hoc. Elas são criadas por clonagem de pipelines modelos ou templates.

Estas pipelines modelos são aquelas mantidas sob controle da matriz para garantir que todas as pipelines criadas estejam aderentes aos padrões definidos.

Existem diversos modelos de pipelines de build e cada um deles atende a uma determinada tecnologia. Ainda há muitos outros a serem criados para tornar cada vez mais compatível as opções de processos de integração contínua no ADS.



6.7.1.4. CRIAÇÃO DA PIPELINE

A ação de criação de pipelines é EXCLUSIVA do perfil SUPORTE e ocorre logo que o time cria uma requisição (ver seção SUPORTE) para incluir ou migrar um módulo de um sistema (ou todo o sistema).

A criação é baseada em um modelo compatível conforme as tecnologias informadas no ato da requisição.

6.7.1.5. CRIAÇÃO DE TAREFAS (TASKS)

Uma tarefa é o bloco de construção para definir a automação em um pipeline. Uma tarefa é simplesmente um script ou um procedimento empacotado.

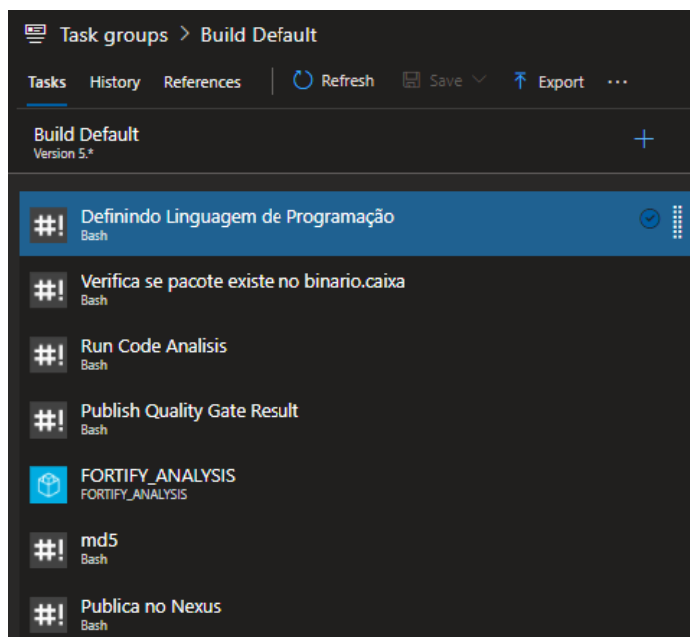
Uma pipeline clássica executa tarefas ou grupos de tarefas de forma sequencial.

6.7.1.6. CRIAÇÃO DE GRUPOS DE TAREFAS (TASK GROUPS)

Tarefas afins podem ser organizadas em grupos, as *Task Groups*.

Uma pipeline é uma estrutura composta por tarefas sequenciadas dentro de uma lógica de automação. Uma pipeline é formada de tarefas e grupos de tarefas.

Na figura abaixo, a *task group* “build default”, que faz parte da pipeline de build do modelo Java-Maven-Build, apresenta as seguintes tarefas: (i) definindo (identificando a) linguagem de programação; (ii) verifica se (a versão) do pacote existe no binário; (iii) Run code Analysis (scan do sonar); (iv) Publish Quality Gate Result (saída do resultado); (v) Fortify Analysis (análise da Task Group do fortify); (vi) hash (verifica integridade do artefato) e (vii) Publica no Nexus.



6.7.1.7. CONTROLE DE VERSÃO DAS TAREFAS E TASK GOUPS

Tarefas e task groups devem ser versionadas, ou seja, em caso de necessidade de corrigir ou melhorar os scripts, não há necessidade de sobrescrição, basta gerar uma nova versão.

Assim, pipelines têm a possibilidade de executar as tasks ou task groups pertencentes as suas pipelines nas versões de suas necessidades ou interesses.

Há uma restrição de alteração para versões desatualizadas. Só é possível alterar os scripts de uma versão atual (caminho único), nunca uma versão anterior (perene).

Alterar, excluir ou desativar tarefas de uma task group ou task groups dentro de outras task groups NÃO afeta as outras versões. Não há um efeito de herança para versões anteriores.

6.7.1.8. NOMENCLATURA DA PIPELINE

A nomenclatura da pipeline obedece ao seguinte padrão:

SISTEMA-módulo-complemento, onde:

SISTEMA é o nome do sistema em letras maiúsculas, módulo é o identificador do módulo do sistema em letras minúsculas, complemento é qualquer informação que o time percebe como válido em letras minúsculas.

Ex:

SIHDG-frontend

SIEVJ-api-portal

6.7.1.9. PERMISSIONAMENTO DA PIPELINE

Uma vez criada a pipeline de build, ele automaticamente herda o permissionamento da pasta do sistema onde ela pertença.

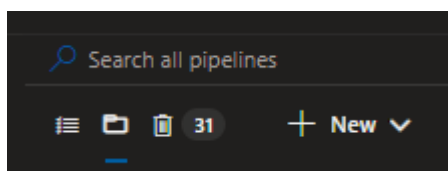
No ato da criação da pasta, caso não exista, o permissionamento é para o TIME e ele deve habilitar as duas últimas opções, (i) view build pipeline e (ii) view builds.

ACCESS CONTROL SUMMARY	
Shows information about the permissions being granted to this identity	
Administer build permissions	Not set
Delete build pipeline	Not set
Delete builds	Not set
Destroy builds	Not set
Edit build pipeline	Not set
Edit build quality	Not set
Manage build qualities	Not set
Manage build queue	Not set
Override check-in validation by build	Not set
Queue builds	Not set
Retain indefinitely	Not set
Stop builds	Not set
Update build information	Not set
View build pipeline	Allow
View builds	Allow

6.7.1.10. LOCALIZAÇÃO DA PIPELINE

Assim que a pipeline for permissionada para um time, todos os perfis ou grupos do time passam a “enxergar” a pipeline e atuar conforme seus papéis.

É possível agora usar o mecanismo de busca para localizar a pipeline. Selecione o ícone da pasta (está com um traço em azul) e escreva o nome do sistema. Agora, todas as pipelines existentes daquele sistema estarão visíveis.



6.7.1.11. EXECUÇÃO DA PIPELINE

Com a pipeline pronta e permissionada, basta uma nova versão criada para poder “rodá-la”.

O acionamento pode ser automático, via política de branch ativada (integração contínua ativada) ou manualmente, agendando-a para uma “fila” ou “queue”.

Assim que a opção de fila estiver ativada, ela aguarda a liberação de um agente para iniciar a esteira.

O dashboard de acompanhamento apresenta uma relação de execuções de uma mesma pipeline, inclusive os identificadores dos builds (BuildID) que estão sendo construídos.

Valida-Java-Maven-Build-AtualizaADS			
History Deleted Analytics			
Commit			Build #
MP	Updated index.jsp, espaço Manual build for Manuel Nunes Pereira	🔒 ✓	56795
LA	Updated index.jsp, espaço Manual build for Lucimar Rodrigues de Araujo	🔒 ✓	56629
MP	Updated index.jsp, espaço Manual build for Manuel Nunes Pereira	✓	56583
MP	Updated index.jsp, espaço Manual build for Manuel Nunes Pereira	⋮ ✗	56540

6.7.1.12. ACOMPANHAMENTO DOS LOGS

Iniciada a esteira, é possível acompanhar as ações e resultados de cada tarefa a cada execução de uma pipeline.

 **#56795: Updated index.jsp, espaço**

Manually run seg. at 10:07 by Manuel Nunes Pereira  sample-java  4f7a708  Retained by release

Logs Summary Tests

Agent job 1
 Pool: Build-Linux · Agent: cadsva@r1x020.intra.caixa.gov.br

-  Prepare job · succeeded
-  Initialize job · succeeded
-  Checkout · succeeded
-  Maven pom.xml · succeeded
-  Recupera versão projeto (groupid:artifactId:type:version) · succeeded
-  Definindo Linguagem de Programação · succeeded
-  Verifica se pacote existe no binário.caixa · succeeded

Para observar os detalhes das tarefas, basta clicar em um passo ou step, como o “Maven pom.xml”

 **Maven pom.xml**

```

1  [section]Starting: Maven pom.xml
2  =====
3  Task           : Maven
4  Description    : Build, test, and deploy with Apache Maven
5  Version       : 3.13.0
6  Author        : Microsoft Corporation
7  Help          : https://docs.microsoft.com/azure/devops/pipelines/tasks/build/maven
8  =====
9  [command]/opt/apache-maven/apache-maven-3.5.3/bin/mvn -version
10 Apache Maven 3.5.3 (3383c7e1f9e9b3cd3d5f580c29cfa999295297; 2018-02-24T16:49:05-03:00)
11 Maven home: /opt/apache-maven/apache-maven-3.5.3
12 Java version: 1.8.0_221, vendor: Oracle Corporation
13 Java home: /usr/java/jdk1.8.0_221/jre
14 Default locale: pt_BR, platform encoding: UTF-8
15 OS name: "linux", version: "3.10.0-1062.9.1.el7.x86_64", arch: "amd64", family: "unix"
16 SYSTEMS_CONNECTION exists true
17 [command]/opt/apache-maven/apache-maven-3.5.3/bin/mvn -f /opt/ads-agent/_work/1041/s/pom.xml clean compile install -Dversion.app=10.1.4-SNAPSHOT
18 [INFO] Scanning for projects...
19 [WARNING] Some problems were encountered while building the effective model for openshift:sample:war:10.1.4-SNAPSHOT
20 [WARNING] 'version' contains an expression but should be a constant. @ openshift:sample:${version.app}, /opt/ads-agent/_work/1041/s/pom.xml, line 7, column 12
21 [WARNING] It is highly recommended to fix these problems because they threaten the stability of your build.
22 [WARNING] For this reason, future Maven versions might no longer support building such malformed projects.
23 [INFO]
24 [INFO] -----< openshift:sample >-----
25 [INFO] building getting-started-tomcat Maven webapp 10.1.4-SNAPSHOT
26 [INFO] -----[ war ]-----
27 [INFO]
  
```

Neste recorte do log, é possível ver que a versão utilizada é a 10.1.4-SNAPSHOT (versão descartável).

6.7.1.13. MONTAGEM DA IMAGEM

No caso de container, é gerado um outro produto que é a criação da imagem (camada) para o OKD para ser referenciada na pipeline de release (subida do ambiente).

6.7.2.PIPELINES DE RELEASE

Seguindo a mesma lógica das pipelines de build, as pipelines de release automatizam tarefas, mas o objetivo aqui é configurar e publicar releases em ambientes virtualizados, promovendo a Publicação Contínua.

A pipeline de release possui a seguinte estrutura e elementos:

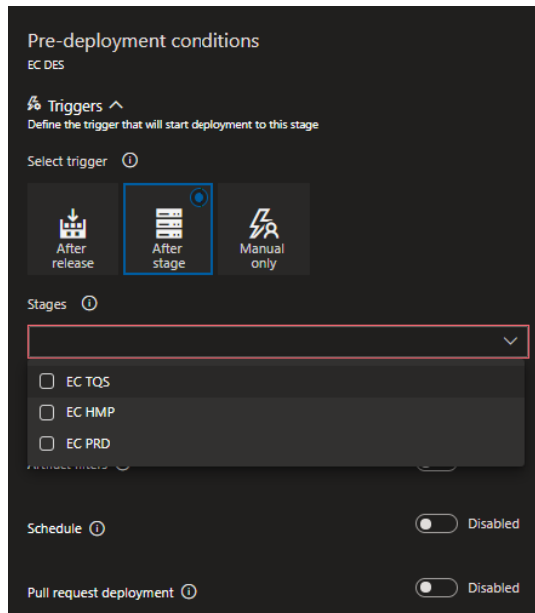


Artefatos: Uma pipeline de release deve apontar para uma pipeline de build que cria e publica os artefatos que serão objetos das novas versões e um repositório de configurações para a configuração e atualização dos ambientes.

Stages: A pipeline de release é composta por estágios ou *stages* que, por sua vez, são compostos por tarefas. Cada estágio tem como finalidade configurar, atualizar e publicar versões da aplicação em cada um dos ambientes desejados, normalmente, os ambientes de DES, TQS, HMP e PRD.

Cada stage tem opções de inclusão de triggers, pré-aprovadores e pós-aprovadores.

Triggers: são condições que, quando atendidas, liberam o fluxo de trabalho. É possível criar triggers entre os stages (quando um termina, o próximo começa automaticamente), agendas, pull request, entre outros gatilhos.



Aprovadores: é possível incluir pessoas ou grupos para pré-aprovar ou pós-aprovar um stage. Por exemplo, mesmo que haja um trigger de ligação entre o stage de TQS e HMP, a inclusão de um pré-aprovador no stage de HMP INTERROMPE o fluxo, aguardando a sua aprovação.

Pre-deployment conditions

EC DES

Triggers 

Define the trigger that will start deployment to this stage

Pre-deployment approvals   Enabled

Select the users who can approve or reject deployments to this stage

Approvers 

 Enter at least one approver.

Timeout 



Approval policies

☐ The user requesting a release or deployment should not approve it

☐ Skip approval if the same approver approved the previous stage 

Link: um stage deve ter associado (link) a ele um grupo de variáveis ou *variable groups* que será utilizado no momento da subida do ambiente e publicação.

6.7.3.VARIABLE GROUPS

São grupos de variáveis que mantêm valores externalizados de parâmetros, urls, senhas, users, entre outros, que servem para serem utilizados pelos stages no momento de configuração de cada um dos ambientes.

É a equipe de SUPORTE que irá criar e linkar a Variable Group com o stage correspondente. No momento da criação compete ao time de desenvolvimento colaborar para que todas as variáveis relevantes da aplicação/módulo estejam mapeadas e seus valores corretamente informados.

Por exemplo, numa arquitetura java, as variáveis incluídas para o determinado ambiente DEVEM ter o mesmo nome que as propriedades (properties) declaradas no standalone.xml para o JBOSS.

Exemplo, a property name "AMBIENTE", deve receber o valor "__AMBIENTE__" (com underscore duplo antes e depois da string)

Uma vez declarada essa propriedade, deve-se adicionar na variable group, o nome da propriedade como variável e o valor pertinente a configuração do ambiente desejado.

Assim, a propriedade receberá externamente (via variable group), os valores adequados a cada configuração de ambiente (via stage).

```
<extension module="org.wildfly.extension.undertow"/>
</extensions>
<system-properties>
  <property name="url_sso_internet" value="__SSO_URL_INTERNET__"/>
  <property name="url_sso_intranet" value="__SSO_URL_INTRANET__"/>
  <property name="AMBIENTE" value="__AMBIENTE__"/>
  <property name="VERSAO" value="__VERSAO__"/>
</system-properties>
```

Name ↑	Value	
AMBIENTE	Usuario01	

Para variáveis cujo valor é secreto, ao clicar no cadeado, o valor passa a ser criptografado. O nome da variável deve ser igual ao da variável no "standalone.xml", sem o ***underline***.

Name ↑	Value	
AMBIENTE	*****	

Atentar que este tipo de variável é mantido pela área de Segurança.

Os arquivos de configuração do ambiente, como o standalone-full-ha.xml (perfil completo para configuração de clustering de VM), standalone-okd.xml (para gestão de container), entre outros, estão localizados nos repositórios de configuração (com as versões controladas).

As variables groups linkadas com os stages de DES e TQS são mantidas pelos times de desenvolvimento. As variables de HMP e PRD são mantidas pelo time de Operações.

6.7.4. TRIGGER PARA INTEGRAÇÃO CONTÍNUA

Para que o processo de geração de artefato (pipeline de build) ocorra de forma automática sempre que houver merge numa determinada pipeline, é necessário configurar o “trigger” da pipeline (por padrão está desabilitado)

Neste caso, deve-se marcar a opção de “enable continous integration” e selecionar o type “include” para a(s) branch(es) desejada(s).

Continuous integration



sample-java
Enabled

Scheduled

+ Add

No builds scheduled

Build completion

+ Add

Build when another build completes

sample-java

- ☒ Enable continuous integration
- ☐ Batch changes while a build is in progress

Branch filters

Type

Branch specification

Include

develop

+ Add

Path filters

+ Add

7. AZURE NA NUVEM (AZURE DEVOPS SERVICE)

O Azure Devops Service é um conjunto de serviços disponibilizados na nuvem que permite utilizar nas esteiras criadas, ambientes, infra, tarefas, repositórios e serviços em geral, de forma transparente e com baixa ou nenhuma customização.

7.1. ORGANIZAÇÃO CAIXA_AZURE

Todos os sistemas/módulos passíveis de irem para a nuvem deverão ser mantidos na organização CAIXA_AZURE.

Nesta organização, cada sistema deverá ser mantido em projetos exclusivos, num modelo conhecido como multiprojetos.

O projeto a ser criado deve ser vinculado única e exclusivamente a um SISTEMA (item de portfólio) da comunidade, ou seja, caso a comunidade tenha 10 siglas distintas sob sua gestão, ela terá 10 áreas de projetos distintos dentro do Azure Devops Service. Por outro lado, um sistema pode ter inúmeros módulos, como SISTEMA-frontent, SISTEMA-backend, entre outros. Todos os módulos do mesmo sistema devem fazer parte da mesma área de projeto.

Em casos em que o mesmo sistema contenha mais de uma tecnologia, por exemplo, SIABC-ios-nativo e SIABC-ios-híbrido, o projeto será o mesmo, mas as divisões serão feitas através de diferenciação dos repositórios e pipelines (ou módulos) de cada tecnologia empregada em cada aplicação.

Os membros de um projeto são adicionados via grupos de AD, portanto, o mesmo grupo de AD pode ser adicionado para várias áreas de projetos pertencentes a mesma comunidade.

7.2. CRITÉRIOS PARA CRIAÇÃO DE PROJETOS NO CAIXA_AZURE

Caso o time tenha interesse que determinado sistema ou módulo use esteiras de automação na nuvem, é necessário se atentar aos seguintes critérios/necessidades:

- capacidade do time para criar e manter pipelines no ambiente de nuvem.
- Arquitetura da aplicação

É importante enfatizar que as esteiras na nuvem são criadas pelo próprio time, conforme necessidades e características da aplicação a ser construída e que a infra básica é suportada pela Microsoft. O suporte CAIXA é limitado ao uso de recursos/serviços, imagens, integração com a rede CAIXA e com o devops.caixa, além de mentoria sobre o modelo de uso, portanto é determinante para o uso na nuvem é que o time tenha **capacidade para criar, executar e ajustar as pipelines de forma autônoma e independente.**

Caso o time não possua esta capacidade ou prefira um modelo com maior suporte deve optar pelo AZURE DEVOPS SERVER, ou seja, a solução *on premise* acessada em devops.caixa.

A GEQTI promove cursos e treinamentos visando dar maior capacidade, maturidade e autonomia devops aos times de desenvolvimento. Para maiores informações, entre em contato com a GEQTI.

7.3. CRIAÇÃO DE PROJETOS NO CAIXA_AZURE

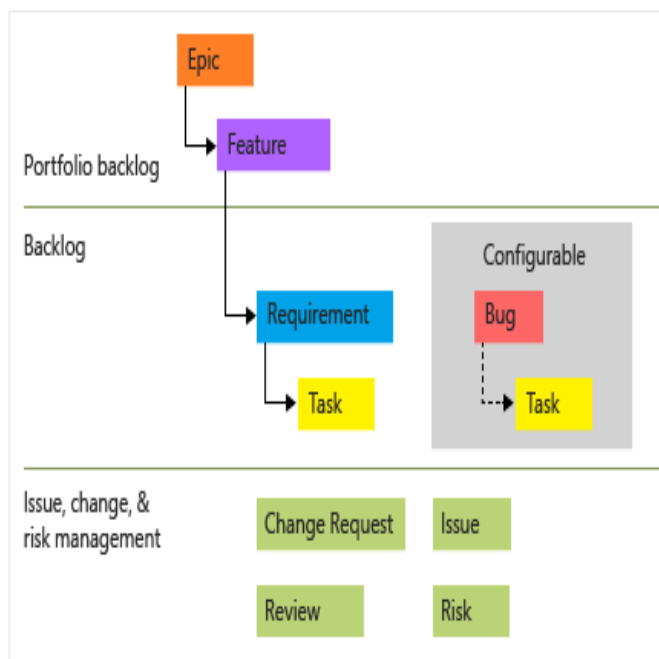
O primeiro passo para o time é criar um projeto na nuvem.

Esta solicitação deve ser encaminhada a equipe de suporte e o nome que o time irá informar deve ser o nome do sistema (conforme SIGLA constante no SIBTI.caixa), que será atribuído ao projeto.

As esteiras criadas neste projeto devem utilizar o padrão de nomenclatura SIGLA-módulo, por exemplo, SIABC-backend e SIABC-frontend.

Este padrão de nomenclatura será usado para apontamento do Project.key e Project.name do Sonar e SSC application name, no Fortify, portanto, manter o padrão irá permitir compatibilidade semântica a todas as ferramentas, facilita a monitoração e a coleta de indicadores.

O Modelo de processo adotado no projeto será o CMMI, por possuir uma maior variedade de tipos de itens de trabalho.

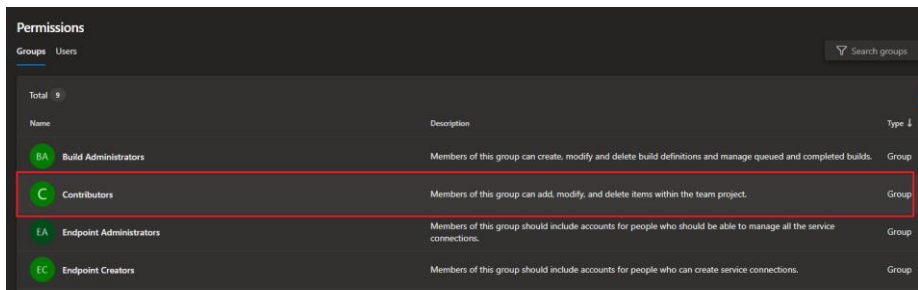


A equipe de suporte irá entregar ao time o projeto solicitado, devidamente configurado, permissionado, um repositório pronto para uso com um readme.md e uma estrutura básica de yml para ser configurado pelo time.

7.3.1.PERFIS & PERMISSIONAMENTO NO CAIXA_AZURE

Por existir integração entre o CAIXA_AZURE e a estrutura interna de AD da Caixa, os grupos de AD pré-existentes ou novos grupos, conforme a necessidade e interesse dos times, poderão ser adicionados nos projetos.

É importante ressaltar que no modelo de multi-projetos, existirá apenas um único papel para os times de desenvolvimento, que será o papel nativo de contribuidor. Este perfil permite pleno acesso e edição aos repositórios, pipelines e demais elementos dentro do projeto. Por uma questão de governança, este perfil terá apenas uma limitação, a exclusão de repositórios e pipelines.



Permissions		
Groups		Users
Search groups		
Total 9		
Name	Description	Type
BA Build Administrators	Members of this group can create, modify and delete build definitions and manage queued and completed builds.	Group
C Contributors	Members of this group can add, modify, and delete items within the team project.	Group
EA Endpoint Administrators	Members of this group should include accounts for people who should be able to manage all the service connections.	Group
EC Endpoint Creators	Members of this group should include accounts for people who can create service connections.	Group

É de responsabilidade do time a criação e manutenção dos grupos de AD que terão permissão de acesso ao dev.azure.com

Para os times de operações, o perfil a ser utilizado é o Release Administrator, que deve ser usado para aprovações nos estágios de HMP e PRD, quando estes forem pertinentes.

O perfil de Project Administrator é reservado exclusivamente para o suporte.

Para demais interessados, um perfil apenas para leitura, o perfil a ser utilizado é o de reader, normalmente um licenciamento “stakeholder”.

Contribuidor	Release Administrator	Project Administrator	Reader
• Perfil padrão para desenvolvedor • Sem delegação	• Perfil padrão para operações	• Perfil Suporte	• Perfil stakeholder

7.3.2.CRIAÇÃO DE REPOSITÓRIOS E PIPELINES NO CAIXA_AZURE

A criação de repositórios e pipelines será realizada pelo próprio time, preferencialmente:

- repositórios exclusivos por pipelines ou módulos,
- uso das boas práticas de controle de versão,
- uso de gitflow ou release flow para gerenciamento de branches,
- uso do pull request nas branches dedicadas para versões candidatas,
- aprovação do pull request pelo GCM/Integrador da comunidade
- uso de triggers para habilitar integração e publicação contínua,
- aprovadores por stages para melhor controle de qualidade e publicações e,
- uso do yaml.

O uso do modelo yaml como forma de edição de pipelines possui vantagens em relação a edição clássico e permite ao time:

- tratar *pipelines*, *jobs*, *stages* e variáveis em scripts mantidos e compartilhados no git,
- controlar e retornar versão anteriores mais facilmente, manipular e alterar valores sempre que necessário,
- comparar as alterações de maneira simples, o que significa facilidade na identificação de correções e ajustes,
- uso de *branches* de desenvolvimento, teste e pull request para versões de pipelines candidatas,
- uso de *tags* para versões estáveis ou indicadas para uso pontual a cada pipeline,
- pleno acesso a histórico, comparação e blame,
- portabilidade total entre projetos distintos, fomentando seu uso de maneira colaborativa para outros sistemas ou times,
- Uso de condições e decisões DURANTE a execução, permitindo uma variedade de possibilidades que atendam às necessidades dos usuários.

7.3.3.TAREFAS DE QUALIDADE

É de responsabilidade no time realizar as chamadas das tarefas que irão realizar a verificação de qualidade e segurança, como as tarefas de execução do SONAR e FORTIFY, conforme definições da GEQTI e GECMI, respectivamente.

A GEQTI irá promover uma análise em tempo de execução para verificar se o time contemplou em suas pipelines as referidas tarefas. Caso não as localize, as pipelines serão quebradas.

As chamadas das tarefas deverão ser introduzidas pelo próprio time em cada pipeline criada e a chamada irá direcionar para um arquivo “template” mantido pela GEQTI com todas as tarefas/steps de qualidade em repositório próprio. As definições técnicas de como inserir esta chamada estão publicadas na wiki da área de Projeto QualidadeCAIXA (na nuvem), assim como outras orientações.

Para evitar problemas é fundamental que o time use apenas uma **única estrutura de pipeline** para cada módulo, ou seja, uma esteira fim a fim, integrando CI/CD. Assim, todas as pipelines sempre contemplarão as tarefas de qualidade, evitando quebrar pipelines que só façam deployment, por exemplo.

Os arquivos mantidos pela GEQTI farão uso de git flow e pull request para controle das versões, portanto, os scripts do decorator e da verificação da Qualidade serão devidamente versionados e aprovados.

7.3.4. ATUAÇÕES NO MODELO DEVOPS PARA NUVEM



PAPÉIS NO DEVOPS NA NUVEM

GEQTI	GESTI	SUPORTE GEPAC	TIMES DEV	TIMES OPS	SEGURANÇA	ARQUITETURA
• Definir as estratégias e diretrizes de DevOps • Definir os modelos de uso para ferramenta DevOps • Definir os modelos de Qualidade • Propor Melhorias e Inovação no uso e nos Modelos • Prestar mentoria ao uso e orientações do modelo em geral (não operacional) • Criação de projetos na nuvem. • Acionamento: RTC (serviço de qualidade)	• Prestar suporte de infraestrutura na nuvem em geral. • Manter o VMSS para garantir integração com o devops.CAIXA. • Acionamento: servicos.caixa	• Prestar suporte operacional em geral (nível 1) nos ambientes NPRD e PRD (conforme o caso)	• Realizar a abertura de chamado para criação de projetos na nuvem • Garantir a execução das tarefas de verificação de qualidade e segurança para toda versão construída. • Criar e manter repositórios e pipelines configuradas e funcionando. • Manter uma boa cobertura de código para testes. • Manter um adequado controle de versão, com uso de pull request e branches adequadas. • Manter as variáveis nos ambientes de NPRD	• Aprovar os stages de PRD sempre quando acionados • Manter as variáveis nos ambientes de PRD • Executar pipelines Environments PRD • Realizar as tratativas necessárias sob seu mandato sobre os eventos decorrentes dos ambientes de PRD.	• Executar políticas de permissionamento • Definir níveis de acesso • Garantir AAA	• Definir e Garantir arquitetura de referência

7.3.5. DIFERENÇAS ENTRE O MODELO ON PREMISE E NUVEM

No modelo on-premise, disponível no devops.caixa, há um único projeto onde TODOS os sistemas são mantidos em um forte modelo de governança que garante segmentação de acessos, controle de ações, padrões, modelos e um suporte operacional atuante para infraestrutura, configurações diversas e incidentes.

No modelo de nuvem, há um projeto para cada sistema, portanto, o CAIXA_AZURE adota um modelo de multiprojetos. Este é um modelo mais flexível, pois os riscos estão restritos a cada projeto e não são irradiados para outros projetos. Além disso, os perfis são nativos, tornando o processo de permissionamento mais simples.

Como um modelo mais localmente restrito de risco operacional, ou seja, o risco por uso fica condicionado a cada projeto, os times na nuvem terão uma liberdade antes restrita: a capacidade de criação e edição de pipelines. É importante frisar que esta flexibilização irá tornar as pipelines divergentes entre si, cada uma criada e mantida num contexto de necessidade por pessoas diferentes e que possuem liberdade de ação. Se por um lado, há benefícios claros pela maior autonomia, por outro, perde-se a capacidade de suporte fora da própria equipe, por não se tratar mais de processos devidamente padronizados.

É importante destacar que não haverá perda no suporte das esteiras na nuvem, apenas não haverá suporte na **edição** das esteiras. Problemas relacionados a criação e edição das esteiras afeitas a aplicação é de responsabilidade **exclusiva** dos times. As tarefas relacionadas a infra continuarão sendo suportadas pelos times de infra, assim como as tarefas de qualidade, pelo time de qualidade e etc.

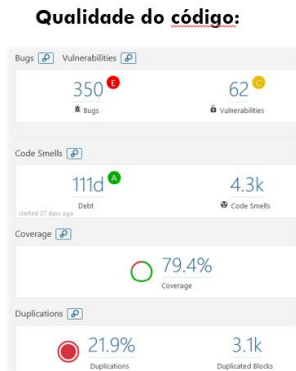
Os padrões de nomenclatura a serem adotados na nuvem é o mesmo do devops.caixa, mas será de responsabilidade do time adotá-los, mantê-los e ajustá-los quando necessário. Não haverá um suporte operacional no mesmo modelo do on-premise, pois cada time terá mais autonomia e deverá ser capaz de garantir a eficiência das suas pipelines.

7.3.6.ORIENTAÇÃO OPERACIONAIS PARA DEVOPS NA NUVEM

Similar a wiki do PROJETO CAIXA no devops.CAIXA para o modelo on-premise, as orientações técnicas são mantidas na wiki do projeto QualidadeCAIXA, na organização CAIXAAZURE (nuvem).

8. TESTES DE QUALIDADE

Esta seção será dedicada a todo o modelo de qualidade integrado com as esteiras DevOps, com especial atenção as integrações com o SONAR, além de citar outras ferramentas, como JMeter, Selenium e Fortify.



8.1. SONAR

O SonarQube é atualmente a ferramenta utilizada nas esteiras para análise estática de código/scripts.

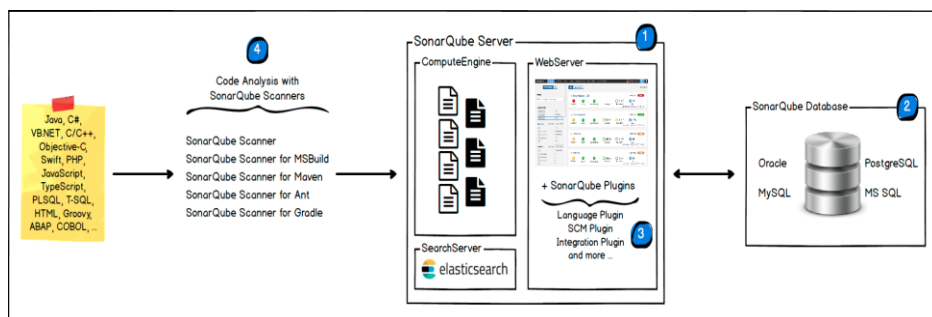
Esta ferramenta possui poderosos dashboards para acompanhamento dos resultados da análise para cada módulo e apontam suas vulnerabilidades, más práticas de codificação e até possíveis bugs.

Um importante aspecto é a possibilidade de utilizar o Sonarqube para “quebrar” a pipeline de build sempre que os critérios mínimos de qualidade não são alcançados.

O SonarQube Corporativo está disponível em: <http://sonar.produtos.caixa.>

8.1.1.ARQUITETURA

A plataforma SonarQube possui 4 componentes:

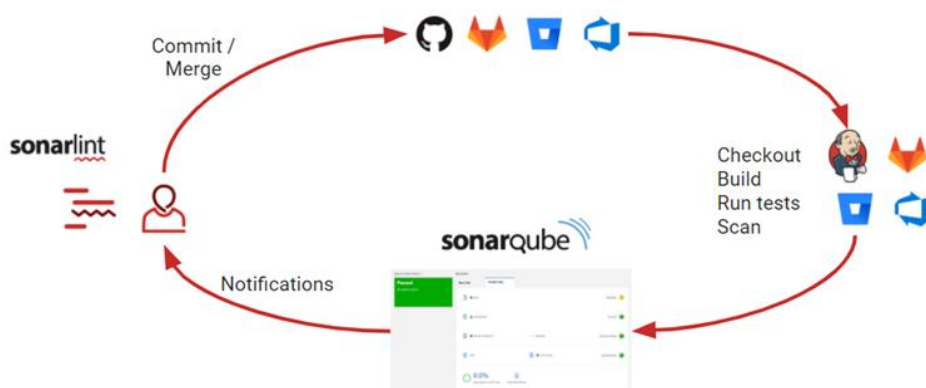


- Um servidor web SonarQube permite que desenvolvedores, gestores e demais interessados naveguem entre os resultados da

análise da qualidade do código obtidas e que configurem essas análises dentro das necessidades estabelecidas.

- Um banco de dados SonarQube para armazenar as configurações e “snapshots” de qualidade de projetos.
- Plug-ins do SonarQube instalados no servidor, incluindo CheckStyle, integração, autenticação e diversos plugins de governança.
- SonarScanner em execução nas pipelines de Build / Pipelines para analisar projetos/módulos em tempo de compilação.

8.1.2.FLUXO DE EXECUÇÃO



Deve-se considerar que todos os códigos **implementados pela CAIXA** (por funcionários ou terceiros) devem sofrer análise pelo Sonar, exceto quando:

- A linguagem em referência não tiver cobertura

- Quando os códigos são gerados por outra ferramenta e com a dispensa autorizada pela GECEQ de vinculação

Arquivos binários e dependências externas de bibliotecas reconhecidas por comunidades não são passíveis de análise pelo SonarQube.

8.1.3.QUALITY GATE

O *quality gate* é uma maneira de estabelecer critérios mínimos de qualidade que devam ser atendidos. O Sonar way é um quality gate nativo e a imagem abaixo apresenta quais são os parâmetros ou condições para que uma análise de código possa ter sucesso ou não.

Quality Gates ⓘ

Coleta de Indicadores

DEFAULT

Modelo de Controle - LEGADOS

Modelo de Referencia - NOVOS

pipeline-ads

SIAPQ Progressivo

SIAPQ Progressivo

SIIFX Batch Progressivo

SIQBR Micro Progressivo

SIQCG Micro Progressivo

SIQPM Micro Progressivo

SIPRG Progressivo

SISPI SIMPI Progressivo

Sonar way

BUILT-IN

Sonar way

BUILT-IN

Conditions ⓘ

Conditions on New Code

Conditions on New Code apply to all branches and to Pull Requests.

Metric	Operator	Value
Coverage	is less than	80.0%
Duplicated Lines (%)	is greater than	3.0%
Maintainability Rating	is worse than	A
Reliability Rating	is worse than	A
Security Hotspots Reviewed	is less than	100%
Security Rating	is worse than	A

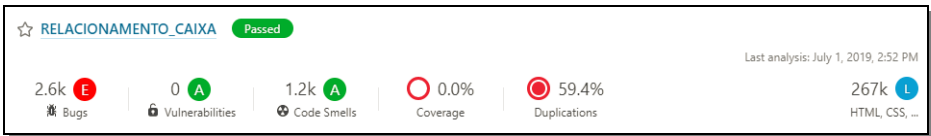
É possível criar ou customizar diversos quality gates e vinculá-los cada um a um projeto ou módulo de interesse. As políticas de Quality Gate são controladas pela GECEQ.

Assim, quando executada a pipeline de build, uma das tarefas é iniciar a verificação do código e, conforme o quality gate associado ao módulo, o resultado pode ser SUCESSO ou não. No caso de não obter sucesso (não passar pelos critérios de quality gate), **a pipeline “quebra” e nenhum artefato ou pacote é gerado.**

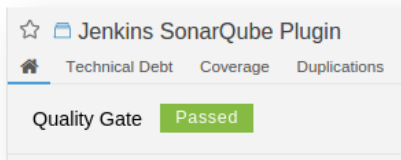
Neste caso, é necessário que o time promova todos os ajustes necessários para gerar uma nova versão e rodar novamente a esteira.

8.1.4.PAINEL DO PROJETO

Na aba PROJETO, o resultado é exibido em destaque na parte superior da página:



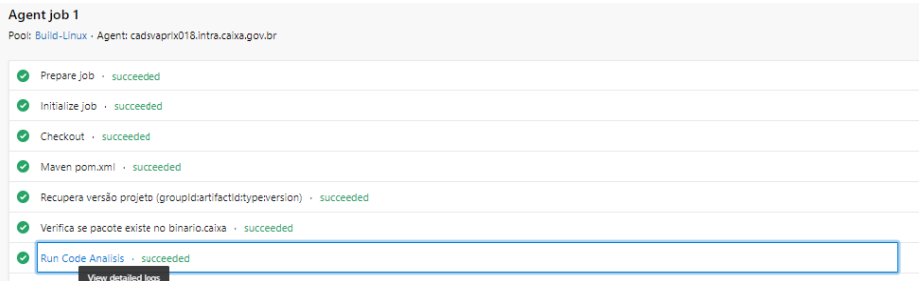
Se o código estiver aderente aos parâmetros definidos no quality gate, uma simples mensagem em verde será exibida (passed).



Caso contrário, informações estarão disponíveis para identificar o que não está aderente, com uma seção para cada condição de erro mostrando qual o valor desejado e possível problema a ser resolvido (issue).

8.1.5.ANALISANDO O LOG

A tarefa Run Code Analysis da task group “Build Default” traz todos os infos e debugs referentes ao processo de análise estática de código realizado no SonarQube.

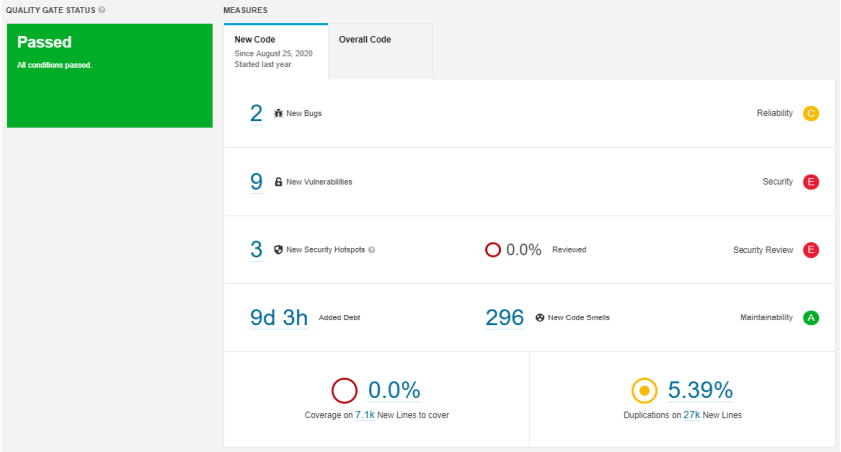


Ao final do log é possível identificar as seguintes informações:

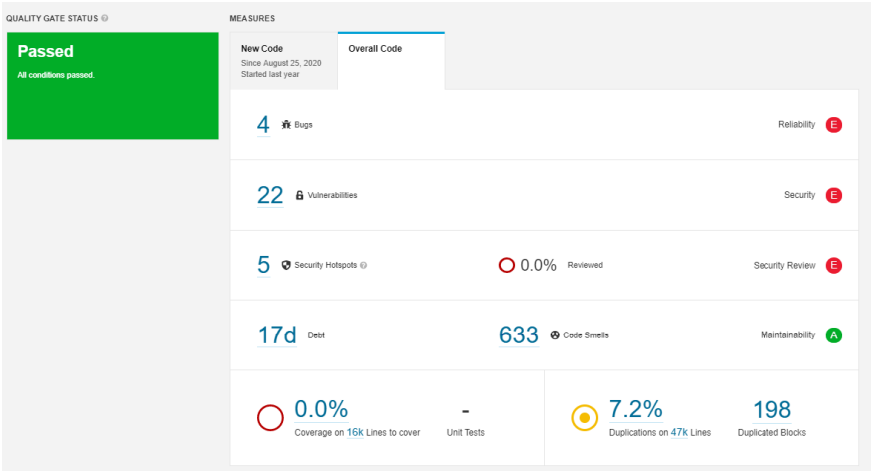
- se a análise obteve sucesso ou não
- qual o endereço para acessar o resultado (you can browse...)
- o id do projeto (neste caso é “sample”)
- o tempo de execução

```
11:48:06.318 INFO: ANALYSIS SUCCESSFUL, you can browse
http://sonar.produtos.caixa/dashboard?id=sample
11:48:06.318 INFO: Note that you will be able to access the updated
dashboard once the server has processed the submitted analysis report
11:48:06.318 INFO: More about the report processing at
http://sonar.produtos.caixa/api/ce/task?id=AX7pCn7taX-yseW20VIO
11:48:06.321 DEBUG: Post-jobs :
11:48:06.324 INFO: Analysis total time: 13.524 s
11:48:06.326 INFO: -----
-----
11:48:06.326 INFO: EXECUTION SUCCESS
11:48:06.326 INFO: -----
-----
11:48:06.326 INFO: Total time: 15.617s
11:48:06.391 INFO: Final Memory: 8M/34M
11:48:06.391 INFO: -----
-----
```

Ao acessar o endereço, surge o resultado da análise do **novo código**,



e uma aba para observar o resultado acumulado ou legado (overall).



8.2. TESTE UNITÁRIO E COBERTURA DE CÓDIGO - JUNIT

Para que haja a análise e avaliação de testes Unitários, o projeto deve ter configurado em seu POM.XML as seguintes configurações:

- Plugin do JUnit;
- Plugin do Maven-Surefire;
- Plugin do Jacoco.

Nas diversas fases de build do Maven, os plugins geram informações que serão usadas pelo Sonar para apontar o percentual do código, coberto de testes unitários, em tempo de build, da aplicação.

Cada plugin tem sua função, conforme descrito abaixo:

- O Junit faz os testes baseando-se nas classes de teste disponibilizadas pelas equipes de desenvolvimento;
- O Surefire gera os relatórios dos testes executados pelo JUnit;
- o JaCoCo faz os cálculos da cobertura de código, baseando-se nos dados gerados pelos plugins apontados anteriormente.

Segue um modelo de configuração que deve ser adicionado ao arquivo POM.XML principal, e do módulo, que fará com que o projeto consiga ter os dados de testes unitários exportados e populados no Sonar.

- Pom Principal;

```
<build>
  <pluginManagement>
    <plugins>

      <plugin>
        <groupId>org.jacoco</groupId>
        <artifactId>jacoco-maven-plugin</artifactId>
        <version>0.8.4</version>
```

```

        </plugin>

    </plugins>
</pluginManagement>

<plugins>
    <plugin>
        <groupId>org.jacoco</groupId>
        <artifactId>jacoco-maven-plugin</artifactId>
        <configuration>
            <destFile>${sonar.jacoco.reportPath}</destFile>
            <append>true</append>
        </configuration>
        <executions>
            <execution>
                <id>agent-for-ut</id>
                <goals>
                    <goal>prepare-agent</goal>
                </goals>
            </execution>
            <execution>
                <id>agent-for-it</id>
                <goals>
                    <goal>prepare-agent-integration</goal>
                </goals>
            </execution>
            <execution>
                <id>jacoco-site</id>
                <phase>verify</phase>
                <goals>
                    <goal>report-aggregate</goal>
                </goals>
            </execution>
        </executions>
    </plugin>
</plugins>
</build>

```

- POM modular

```

<dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
</dependency>

```

Ao final do processo de build tem-se como saída os relatórios do Surefire e Jacoco, que serão usados pelo task group do Sonar para análise e exportação de dados, tanto de código quanto de testes unitários:

- Surefire: diretório_build/modulo/target/surefire-reports/
- JaCoCo: diretório_build/target/jacoco.exec e diretório_build/target/jacoco.xml

Para aplicações que usam Java, será necessário ajustar a task Java-Build, deve ser editada a pipeline de build da aplicação, para versão igual ou superior à 5.x.

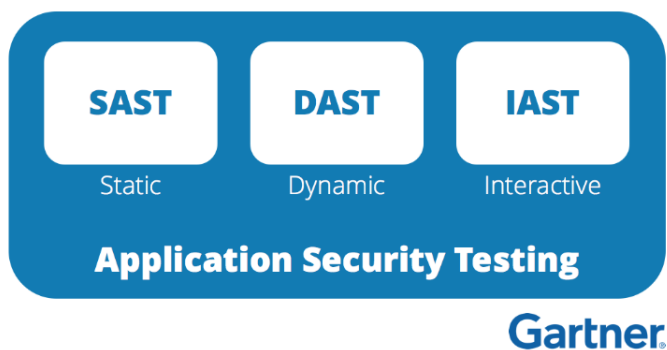
Caso a aplicação utilize outras formas de implementação de testes unitários, será necessária a troca desta informação, diretórios destino dos relatórios, nas variáveis de build do projeto, em conformidade com a tecnologia empregada. A versão atual do Sonar exige o envio do arquivo JACOCO.XML ao mesmo, para processamento dos testes unitários.

8.3. TESTES DE SEGURANÇA

Ferramentas AST são um dos pilares de qualquer prática de segurança de aplicativos.

É importante que os aplicativos devam ser desenvolvidos de forma segura desde o início, e se pensarmos em termos de automação do processo de desenvolvimento, implica em automatizar testes de segurança de forma incremental e contínua a cada nova execução de pipeline.

As ferramentas AST são de três tipos: SAST, DAST e IAST



Ferramentas SAST tem como objetivo identificar as vulnerabilidades no seu código-fonte antes de ele ser publicado. É como uma revisão direta do código-fonte. Para isso são usadas técnicas de análise de código estático para procurar problemas sem precisar executar o código.

Com isso, o SAST consegue encontrar problemas com antecedência, antes da implantação e, por estar agindo no código, pode dar informações detalhadas à equipe para que os ajustes sejam feitos.

Mas, também é preciso estar atento. Por não executar o código, o SAST pode trazer algum nível de imprecisão. E isso pode acabar resultando em falsos positivos ou falsos negativos.

Ao contrário dos SASTs, os DASTs realizam análises de caixa preta do aplicativo, o que significa que eles não têm acesso ao código ou aos detalhes de implementação. Os DASTs examinam apenas as respostas da aplicação a uma bateria de testes projetados para destacar vulnerabilidades. Eles são, em suma, um scanner de vulnerabilidade .

O Interactive Application Security Testing (IAST) é uma combinação de um SAST e um DAST. As ferramentas de segurança IAST oferecem as vantagens de uma visualização estática, porque acessam o código-fonte, e também as vantagens de uma abordagem de scanner de vulnerabilidade, pois percebem o comportamento do aplicativo durante o tempo de execução.

Atualmente, nas esteiras do devops.caixa, há uma tarefa que executa a análise do tipo SAST através da ferramenta Fortify.

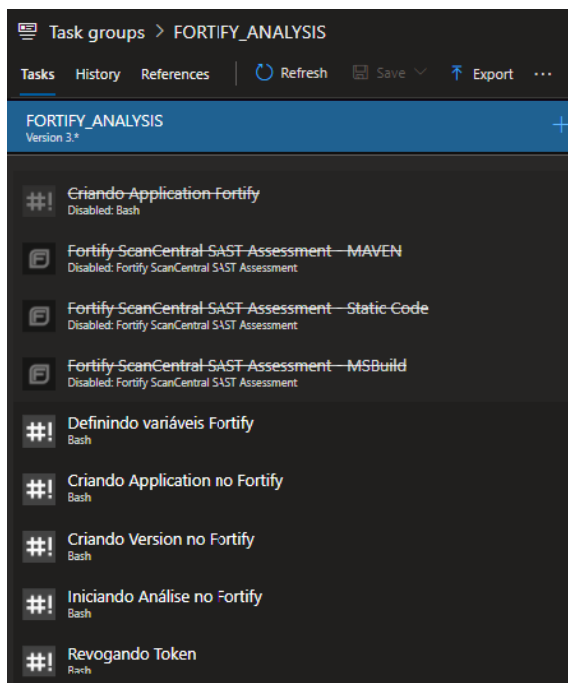
8.4. FORTIFY

O Fortify Static Code Analyzer é uma ferramenta SAST e que possui conjunto de analisadores de segurança de software que pesquisam violações de regras e diretrizes de codificação específicas de segurança em várias linguagens.

Similar a tarefa de análise do SonarQube, violações dos seus parâmetros podem causar a quebra da pipeline de build.

Atualmente, a área responsável por definir as suas políticas é a GECMI.

Abaixo, uma visão de uma task group do Fortify.



9. REPOSITÓRIO DE BINÁRIOS

O Nexus é o gerenciador de repositórios para binários utilizado pelas esteiras do devops.caixa, e é acessado de forma centralizada pela url: `binario.caixa`.

Um gerenciador de repositório desempenha um papel essencial, permitindo que seja possível gerenciar todos os componentes, como dependências, pacotes, snapshots, releases ou imagens, que fazem parte do processo de desenvolvimento.

É importante ressaltar que repositórios Nexus e os repositórios git são complementares e EXCLUDENTES.

Nos repositórios git não é permitido manter nenhum componente, apenas arquivos de texto plano, como os códigos-fonte. Quando há a necessidade de se utilizar uma biblioteca ou dependência qualquer no momento de compilação/empacotamento, ela deve estar exclusivamente no NEXUS (hosted ou proxy).

Repositórios podem ser de acesso público, privado ou controlado. São exemplos de repositórios:

- repositório hospedado de um projeto ou sistema;
- repositório central , também conhecido como Maven Central;
- Outros.

Em seu núcleo, um gerenciador de repositório pode criar os seguintes tipos de repositório:

- repositório privado (hosted) para nossos snapshots
- repositório privado (hosted) para nossos releases
- repositório proxy apontando para Maven Central
- repositório do tipo grupo, para prover todos os tipos de repositórios em uma única URL

Usando essas funções principais, um gerenciador de repositório se torna a plataforma para armazenamento de componentes proprietários (de uso exclusivo da CAIXA) ou de código aberto a serem utilizados ou produzidos pelos times de desenvolvimento.

Com um gerenciador de repositório centralizado, é possível controlar completamente o acesso e a implantação de todos os componentes a partir de um único local, além de examinar e acompanhar o que é produzido por seus sistemas de compilação.

Um gerenciador de repositório também permite proteger uma conexão com repositórios remotos, conforme uma política de acessos externos estabelecida, garantindo que seu uso não seja exposto publicamente. Portanto, o gerenciador de repositório permite fornecer, armazenar, compartilhar e implantar componentes com eficiência.

Em vez de baixar diretamente de repositórios públicos, downloads repetidos ou distribuição manual, o uso de um gerenciador de repositório faz dele o ponto central de acesso e gerenciamento para qualquer uso de componente em seu ciclo de vida de desenvolvimento de software.

Essa função obriga aos times estabelecer uma convenção mínima de um modelo usual para estruturação dos repositórios tornando mais fácil compreender como os componentes estão armazenados, conforme um modelo de organização previamente definido (ver seção sobre organização). Além disso, uma estrutura de acesso eficiente facilita:

- integrar com sistemas de segurança externos, como LDAP;
- ver metadados do componente;
- itens internos do host não disponíveis em repositórios remotos;
- controlar o acesso autenticando e autorizando conjuntos específicos ou grupos de usuários.

9.1. COMPONENTES

No nexus, o termo **componente** descreve itens como dependências, pacotes, bibliotecas, binários, contêiner ou qualquer outro recurso produzido ou usado pelo aplicativo de software em desenvolvimento (ou manutenção).

Os componentes fornecem todos os blocos de construção e recursos que permitem que os times de desenvolvimento criem ou mantenham aplicativos.

Milhões de componentes existem e mais são criados todos os dias por comunidades de código aberto, por fornecedores proprietários ou pelos próprios times.

Um dos muitos benefícios do desenvolvimento de software baseado em componentes é a redução no retrabalho. Por exemplo, se um desenvolvedor criar um widget de calendário e adicioná-lo a um repositório público, esse componente poderá ser reutilizado, sem que uma equipe precise construí-lo a partir do zero, permitindo a eficiência no processo de desenvolvimento.

Normalmente, os componentes são arquivos compactados, compostos de uma grande variedade de tipos de arquivos, como:

- Código de byte Java em arquivos de classe;
- Arquivos de objeto C;
- arquivos em texto puro (plain text), por exemplo, arquivos XML, código JavaScript, HTML, CSS
- arquivos binários, como imagens, vídeo e som;
- Java .jar, .war, extensões de arquivos .ear
- extensões de arquivo .zip .tar.gz
- formatos de pacote como nupkg, RPM e gem
- formatos executáveis como arquivos .exe, .sh ou arquivos APK do Android

Um gerenciador de repositório permite montar componentes remotos e adicionar seus próprios componentes internos para criar aplicativos. Em um gerenciador de repositório, os componentes são identificados por um conjunto de valores específicos definidos pelo formato. Por exemplo, os componentes do Maven usam um conjunto genérico de valores de coordenadas chamado grupo, ID de artefato e versão.

9.1.1.INSTALANDO JDBC DRIVERS DISPONÍVEIS NO BINARIO.CAIXA

A instalação de JDBC Drivers que não estão disponíveis por padrão nas imagens Docker JBOSS EAP, requer que o modulo esteja hospedado nos repositórios do binario.caixa:

<http://binario.caixa:8081/repository/releases> (Para modules criados na Caixa) <http://binario.caixa:8081/repository/thirdparty> (Para modules de terceiros)

Para fazer o download do module você precisara dos seguintes parâmetros que serão passados automaticamente para o comando `mvn org.apache.maven.plugins:maven-dependency-plugin:2.9:get:`

- `group_id`
- `artifact_id`
- `version`
- `extension`

Conforme descrito na wiki do devops.caixa, seção Drivers de Banco de Dados, deve-se criar um arquivo chamado `jboss-modules` no repositório do seu código fonte, este arquivo deve conter o modelo abaixo:

`group_id:artifact_id:version:extension`

9.1.2.DEPLOYMENTS ADICIONAIS JBOSS

A instalação de Deployments adicionais que não estão disponíveis por padrão nas imagens Docker JBOSS EAP, requer que o artefato esteja hospedado binario.caixa nos repositórios:

`http://binario.caixa:8081/repository/releases` ou

`http://binario.caixa:8081/repository/thirdparty`

Para fazer o download do artefato você precisara dos seguintes parâmetros que serão passados automaticamente para o comando `mvn org.apache.maven.plugins:maven-dependency-plugin:2.9:get`:

- `group_id`
- `artifact_id`
- `version`
- `extension`

Crie um arquivo chamado `jboss-deployments` no repositório do seu código fonte, este arquivo deve conter o modelo abaixo:

`group_id:artifact_id:version:extension`

`mysql:mysql-connector-java:8.0.21:jar`

9.2. NOMENCLATURA DE REPOSITÓRIOS

Na criação de repositórios pelo administrador, a nomenclatura dos repositórios deve seguir o seguinte padrão:

caixa-repo-xx-release
caixa-repo-xx-snapshot
caixa-repo-xx-zz

xx representa a unidade e zz representa um repositório extra.

Segue imagem para ilustrar o modelo de nomenclatura dos repositórios:

	caixa-repo-br-releases
	caixa-repo-br-snapshots
	caixa-repo-br-third-party
	caixa-repo-loterias-releases
	caixa-repo-loterias-snapshots
	caixa-repo-loterias-third-party
	caixa-repo-rj-releases
	caixa-repo-rj-snapshots
	caixa-repo-rj-third-party
	caixa-repo-sp-releases
	caixa-repo-sp-snapshots

9.3. CAIXA-GROUP

O grupo de repositório caixa-group é para manter binários construídos pela CAIXA e deve incluir como membros, todos os repositórios que não possuam restrição de acesso dentro da rede caixa de forma transversal de seus componentes.

Para bibliotecas de terceiros, deve-se utilizar o repositório *thirdparty*.

9.3.1. POPULAR REPOSITÓRIOS

É importante que os administradores e uploaders popularem os repositórios hospedados (hosted) do NEXUS, de tal forma a atender as necessidades de uso de componentes pelos times das unidades vinculadas.

Deve-se evitar o uso de componentes como restrição de licenças, falhas de segurança, baixa compatibilidade, entre outros.

Quando houver a inclusão de um novo componente, recomenda-se informar ao comitê, via caixa-postal, de forma sucinta, a definição do novo componente e o seu objetivo, para conhecimento de todos.

9.3.2. ACESSO EXTERNO

O Nexus deverá, preferencialmente, utilizar exclusivamente os repositórios hospedados, com acesso externo controlado.

É necessário que os repositórios busquem atingir um nível de completude que evite acessos externos desnecessários.

Grupos montados com repositórios externos para atender necessidades específicas devem ser solicitados diretamente aos administradores de vinculação.

O controle de acesso externo é necessário para evitar riscos no uso de componentes, como restrição de licença, falhas de segurança, baixa compatibilidade, entre outros.

9.3.3.EXPURGO

Para uma utilização racional do espaço físico a ser utilizado pelo NEXUS, recomenda-se que se mantenha arquivado entre três a cinco releases do sistema. Entende-se como releases, as versões estáveis devidamente testadas e prontas para ir para produção.

Para as versões SNAPHOTS, o prazo é de três meses.

É importante ressaltar que esta política pode ser flexibilizada, caso a área de suporte identificar a necessidade de manter um número maior de versões por características especiais do sistema.

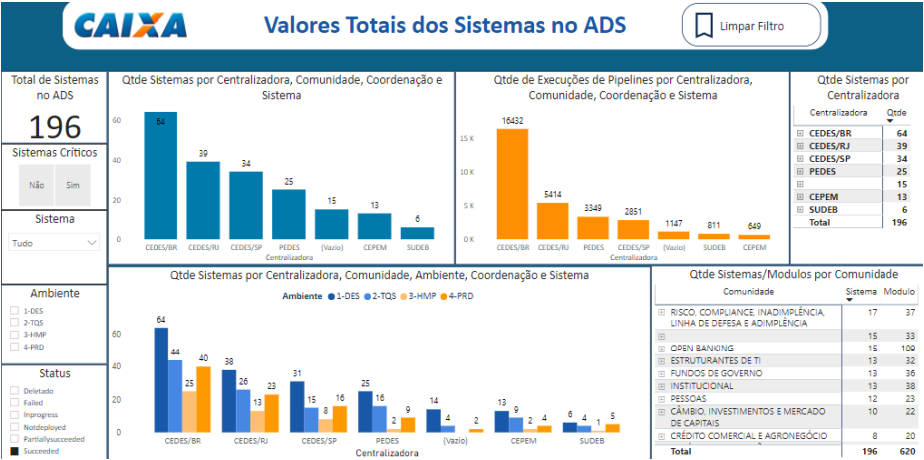
10.INDICADORES & INDUTOR

Esta seção será dedicada ao gerenciamento do uso das esteiras de DevOps, em especial os indicadores e o indutor.

10.1. PLANO DE INDICADORES DO ADS

O Plano de Indicadores do ADS apresenta um relatório composto de diversas abas e pode ser acessado navegando em:

novobi.caixa / Relatórios Públicos / 5222 – GECEQ / GECEQ03 / Plano de Indicadores do ADS



Valor Acumulado: esta aba informa os valores acumulados (total) de sistemas que possuem pelo menos um módulo no devops.caixa, além de informar a quantidades de sistemas por comunidade, centralizadora e coordenação, total de publicação por ambiente com sucesso ou não.

Valor Incremental Mensal: esta aba informa os valores incrementais, mês a mês e suas curvas de evolução dos sistemas incluídos/migrados.

Valor Incremental Mensal (módulos): esta aba informa os valores incrementais, mês a mês e suas curvas de evolução, similar a aba anterior, porém por módulos.

Execuções de Pipelines: esta aba informa a quantidade de execuções de pipelines por comunidade, centralizadora, sistema e aqueles que foram executados com sucesso ou não.

Sistemas por Ambiente: esta aba informa em detalhes a evolução da adesão de sistemas ao Modelo Corporativo, mês a mês, por ambiente.

10.2. INDUTOR DEVOPS

O indutor de DevOps tem como objetivo promover cada vez mais a adesão de sistemas ao Modelo Corporativo e disponibilizar os benefícios que este modelo oferece.

Além disso, o objetivo é dar um indutor de evolução nível a nível, promovendo o uso pleno dos processos de Entrega Contínua a todos os módulos compatíveis do sistema.

Este indutor é baseado em extrações rotineiras da base do ADS em devops.caixa e identifica módulo a módulo, sistema a sistema, quais execuções de pipelines de build e release foram executadas com sucesso e em quais ambientes, e disponibiliza para consulta no Plano de Indicadores do ADS.

Até o nível 3, a esteira afere de forma totalmente automatizada, através de mecanismos próprios, os resultados de avaliação de maturidade. A partir do nível 4, é incluído o contexto de qualidade.

Na aferição dos níveis 4 e 5, o indutor mescla informações automatizadas com informações inseridas por analistas, tanto para informar se a configuração das pipelines está adequada para buscar, capturar e tratar informações em ferramental próprio de verificação da qualidade, como também deve ser informado se os resultados obtidos do referido ferramental estão plenamente qualificados.

Essa mescla na aferição dos níveis 4 e 5 permite ao indutor a efetividade em afirmar que:

- A esteira contém o indicativo para a quebra de build, e o faz;

- As ferramentas que determinam os atributos de qualidade estão aferindo os atributos previstos para a tecnologia aplicada ao sistema;
- Os atributos de qualidade aferidos estão cumprindo os critérios de qualidade determinados pela CAIXA.

Entende-se por informações qualificadas, aquelas em que os consultores responsáveis da ferramenta indicam que o processo de verificação está devidamente ajustado e atinge ou não os critérios mínimos determinados pela CAIXA.

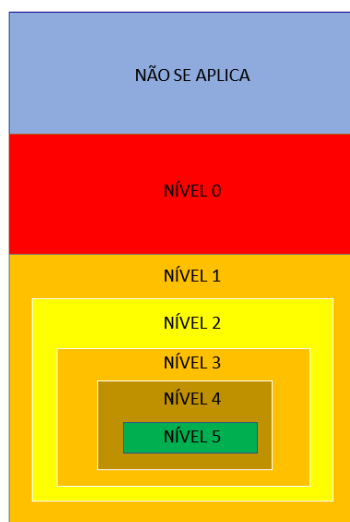
10.2.1. NÍVEIS

Um sistema que pertence ao portfólio formal da TI (sibti.caixa), pode ser relacionado aos diversos níveis do INDUTOR DEVOPS.

NÃO SE APLICA - Do total de sistemas que pertençam ao portfólio e que estão cadastrados no sibti.caixa, parte deles ainda não são compatíveis com as tecnologias atualmente disponíveis para esteiras DevOps (devops.caixa).

NÍVEL 0 – Dos sistemas passíveis de migrarem para esteiras DevOps, parte deles ainda ou não migraram ou ainda não geraram artefatos de forma automática na esteira DevOps.

Níveis 1, 2, 3, 4, 5 – São níveis que representam sistemas, cujas esteiras criaram artefatos ou publicaram releases com sucesso.



10.2.2. NÍVEL 1

O sistema possui uma esteira que tenha realizado uma publicação com sucesso em um ambiente de Desenvolvimento.

Objetivo

Garantir que o sistema esteja configurado no <http://devops.caixa> e que tenha realizado um processo de integração contínua e publicação com sucesso em ambiente de desenvolvimento em pelo menos uma esteira.

10.2.3. NÍVEL 2

O sistema possui pelo menos uma entrega em produção com sucesso

Objetivo

Garantir que o sistema esteja configurado no <http://devops.caixa> e que tenha realizado em pelo menos uma esteira, um processo de entrega contínua com publicação em produção com sucesso.

Regras para estar no nível 2

- O sistema deve estar no nível 1

10.2.4. NÍVEL 3

O sistema possui a maioria das esteiras realizando entregas em todos os ambientes previstos com sucesso

Objetivo

Garantir que o sistema esteja configurado no <http://devops.caixa> e que tenha realizado entregas em produção com sucesso para a maioria das esteiras (50% ou mais) em todos os ambientes previstos.

Regras para estar no nível 3

- O sistema deve estar no nível 2
- O sistema deve ter pelo menos a metade do quantitativo de esteiras publicando com sucesso em produção
- O sistema deve ter publicado com sucesso em todos os ambientes previstos (como TQS e HMP, quando houver)

10.2.5. NÍVEL 4

Objetivo

Garantir que o sistema esteja configurado no <http://devops.caixa> e que tenha realizado entregas em produção com sucesso para a maioria das esteiras (50% ou mais) publicando em todos os ambientes previstos, com **todas as tarefas de qualidade previstas**.

Até o nível 3, todas as informações são extraídas, capturadas e transformadas automaticamente. Para aferir o nível 4, faz-se necessário agregar as seguintes informações de analistas responsáveis:

- Se a esteira possui tarefas configuradas para capturar as informações em ferramental próprio de verificação da qualidade e aplicar seus resultados, quando necessário.
- Se as informações capturadas do ferramental próprio de verificação da qualidade estão devidamente ajustadas e atinge ou não os critérios mínimos determinados pela CAIXA.

Regras para estar no nível 4

- O sistema deve estar no nível 3
- O sistema deve estar com pelo menos metade das suas esteiras com tarefas de qualidade devidamente configuradas por cada tipo de ferramenta de verificação prevista.
- As informações capturadas na esteira devem estar devidamente ajustadas com os critérios mínimos determinados pela CAIXA.

10.2.6. NÍVEL 5

Objetivo

Garantir que o sistema esteja configurado no <http://devops.caixa> e que tenha realizado entregas em produção com sucesso para a todas as esteiras, publicando em todos os ambientes previstos, com todas as tarefas de qualidade previstas.

Regras para estar no nível 5

- O sistema deve estar no nível 4

10.2.7.RELATÓRIOS DE NÍVEL DE MATURIDADE

É possível acompanhar a quantidade e evolução dos níveis de maturidade de DevOps por comunidade e centralizadora

Nível de Maturidade: esta aba informa o total de sistemas por cada nível de maturidade, por comunidade e centralizadora.

Evolução do Nível de Maturidade: esta aba informa a evolução mensal das curvas de maturidade.

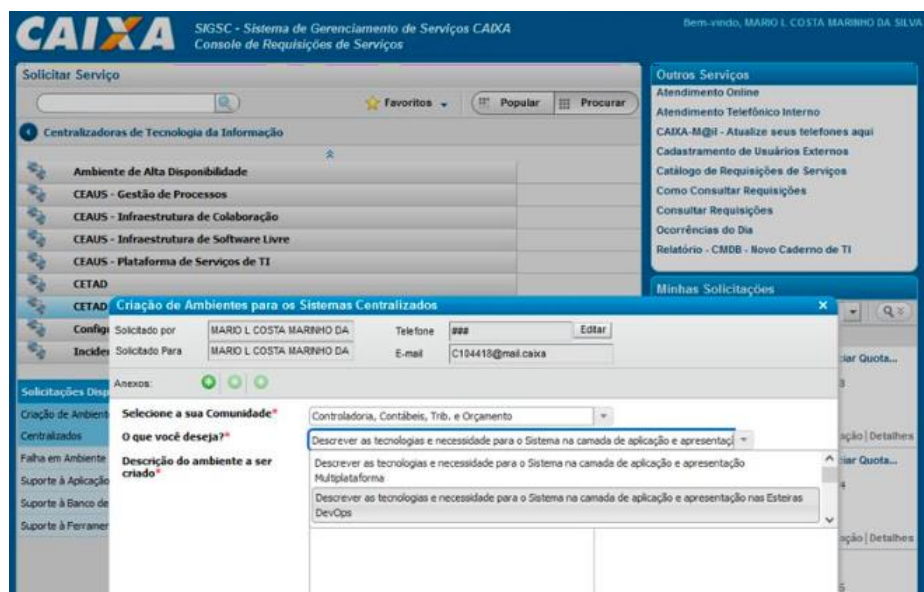
11. SUPORTE

Esta seção será dedicada a orientar as equipes a solicitar o suporte ao Modelo Corporativo de DevOps.

11.1. REQUISIÇÃO DE INCLUSÃO OU MIGRAÇÃO DE SISTEMA NO ADS:

A solicitação é no [servicos.caixa](https://servicos.caixa.gov.br).

Navegue em Tecnologia da Informação / Centralizadores de Tecnologia da Informação / CETAD – Suporte Não Produção.



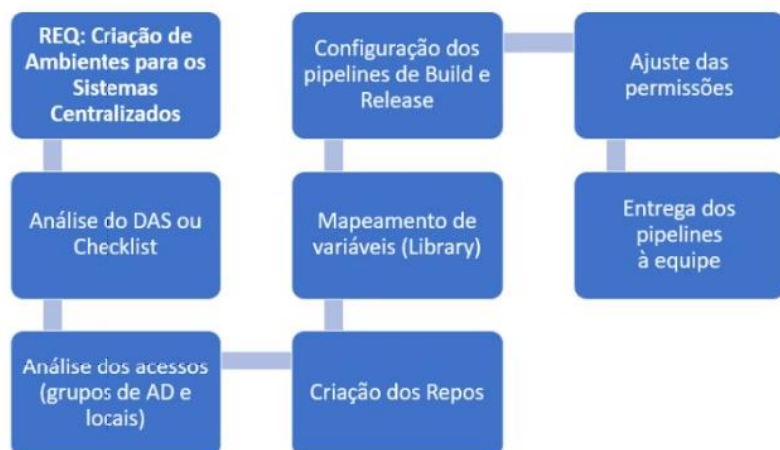
No momento de solicitação do serviço de inclusão/migração dos módulos devem ser informados:

- 1) a comunidade,
- 2) a sigla do sistema,

- 3) o(s) Módulo(s),
- 4) a Fábrica de Software que suporta o sistema,
- 5) pessoa para contato,
- 6) o(s) Repositório(s) Git de origem (no caso de migração),
- 7) as tecnologias utilizadas pelos módulos e
- 8) se o uso é para Integração Contínua ou Entrega Contínua (ciclo completo). Se o uso for para Entrega Contínua:
- 9) a compatibilidade de uso para container (ou VM)
- 10) Documento de Arquitetura (ou similar) que informe:
 - endereços dos servidores (lista de backend's),
 - usuários para acesso aos ambientes de DES, TQS, HMP,
 - driver's que serão utilizados,
 - configurações de Data Sources e
 - demais configurações necessárias que contemplará cada um dos ambientes.

Após a abertura de requisição do serviço, um técnico especializado entrará em contato com a pessoa indicada para proceder o início do processo de inclusão/migração.

Fluxo de Atividades da CETAD:



11.2. SOLICITAÇÃO DE INCLUSÃO DE SISTEMA/MÓDULO NA NUVEM

Na necessidade de criação de esteira uso da esteira no dev.azure.com, deve-se fazer a requisição diretamente à CESTI.

Neste momento, apenas projetos Apple IOS podem usar a infraestrutura da nuvem MS.

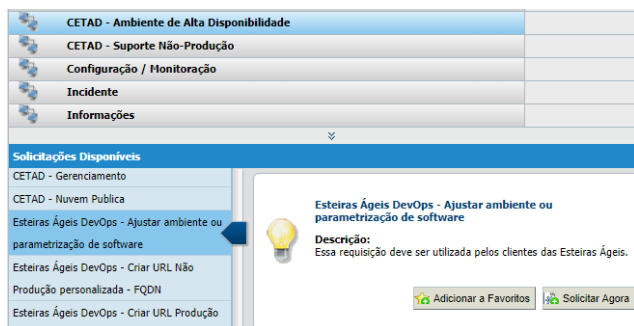
No servicos.caixa, deve ser aberta uma requisição para a equipe da CESTI, com o seguinte fluxo:

Tecnologia da Informação e Comunicação-->

Centralizadoras de Tecnologia da Informação-->

Cetad - Ambiente de Alta Disponibilidade -->

Esteiras Ágeis DevOps - Ajustar ambiente ou parametrização de software



Esteiras Ágeis DevOps - Ajustar ambiente ou parametrização de software

Solicitado por: MARCELO A SANTANA | Telefone: ### | Editar

Solicitado Para: MARCELO A SANTANA | E-mail: C135738@corp.caixa.gov.br

Anexos: + + +

Informar Ambiente*

PRD

Nome do serviço ou sistema*: SIGLA-IOS

Janela do Atendimento*: 09/12/2022 18:32

Prazo final da janela do atendimento*: 12/12/2022 00:00

Haverá previsão de indisponibilidade durante a execução do serviço?: ☐ Sim ☒ Não

Contato do responsável pela validação do ambiente após alteração*: 61 99999-9999

Descrição da solicitação*

SIGLA-IOS- Criação de projeto IOS no ambiente de nuvem da MS

Solicito a criação do projeto para o app mobile - SIGLA-IOS no ambiente de nuvem da MS.

Em anexo, segue o arquivo SpecDeployIOS.docx, que contém as informações básicas de configurações de projetos.

Informar formas de contato*

Teams, telefone

Sumário | Salvar como Rascunho | Enviar

11.3. SOLICITAÇÃO DE ACESSO AS FERRAMENTAS OKD, GRAFANA E GRAYLOG

A solicitação as demais ferramentas de DevOps é via SIGAL em: novoacessologico.caixa/

O grupo de solicitação é: IF_DEVOPS_DO_CEDSXX

11.4. SUPORTE A FERRAMENTAL DEVOPS

Essa opção deve ser utilizado para opção de Produtos relacionados às Esteiras DevOps, tais como: Nexus-binarios.caixa, Fortify, JUnit, Selenium, Graylog, Grafana, Kafka, Datagrid, Kibana, OKD, NFS e Sonar.

Navegue em Tecnologia da Informação / Centralizadores de Tecnologia da Informação / CETAD – Suporte Não Produção.

Suporte à Aplicação Multiplataforma			
Solicitado por	MANUEL N PEREIRA	Telefone	6134486666 Editar
Solidotado Para	MANUEL N PEREIRA	E-mail	C056898@mail.caixa
Anexos:   			
O que você deseja?*		Suporte aos produtos utilizados nas Esteiras DevOps v	
Atenção		Essa opção deve ser utilizado para opção de Produtos relacionados às Esteiras DevOps, tais como: GitLab-Fontes.caixa, Nexus-binários.caixa, Fortify, JUnit, Selenium, Graylog, Grafana, Kafka, Datagrid, Kibana, OKD, NFS e Sonar.	
Qual o nome do produto?*			
Qual o ambiente*			
Qual o nome do Sistema?			
Selecione a sua Comunidade*			
Formas de contato*			
Descrição da necessidade*			